ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Membre de EMSI - HONORIS UNITED UNIVERSITIES

CHU
المركز الاستشفائي الجامعي ابن سينا
CENTRE HOSPITALO-UNIVERSITAIRE IBN SINA

CHU Ibn Sina — Rabat

RAPPORT DE PROJET DE STAGE

Filière Ingénierie Informatique — 3ème année

E-SANTÉ IBN SINA

PLATEFORME INTELLIGENTE DE GESTION CENTRALISÉE DES SOINS

- Réalisé par : Rania Lamsakhar
  Kawtar Chemaou
- Filière : 3IIR11
- Encadré par : M. Yassine Sembati

Chef de Service de Management des Systèmes d'Information — CHU Ibn Sina

Année Universitaire : 2025 - 2026

Table des matières :

Dédicaces **2**

Remerciements **3**

Résumé **4**

Abstract **5**

1 Contexte général du projet **6**
Introduction . . . 6
1.1 Présentation générale du projet . . . 6
1.1.1 Contexte du projet . . . 6
1.1.2 Problématique . . . 6
1.1.3 Objectifs du projet . . . 7
1.1.4 Solution proposée . . . 7
1.2 Étude de l'existant . . . 7
1.2.1 Analyse des solutions existantes . . . 7
1.2.2 Limites des solutions actuelles . . . 8
1.2.3 Valeur ajoutée d'E-Santé Ibn Sina . . . 8

2 Analyse et conception de la plateforme E-Santé Ibn Sina **9**
Introduction . . . 9
2.1 Analyse des besoins . . . 9
2.1.1 Périmètre du projet . . . 9
2.1.2 Acteurs de la plateforme . . . 9
2.1.3 Diagrammes des cas d'utilisation . . . 10
2.1.4 Diagrammes de séquence . . . 13
2.1.5 Diagramme de classes . . . 24
2.2 Conception de la base de données . . . 25
2.2.1 Modèle conceptuel . . . 25
2.2.2 Liste des tables de la base de données . . . 26

3 Réalisation et interfaces de l'application **28**
Introduction . . . 28
3.1 Environnement de développement . . . 28
3.1.1 Environnement matériel . . . 28
3.1.2 Environnement logiciel . . . 28
3.2 Description des besoins techniques . . . 29
3.2.1 Architecture générale de l'application . . . 29

3.2.2 Technologies utilisées . . . 29
3.2.3 Gestion des erreurs d'authentification et alternatives . . . 30
3.2.4 Gestion de la perte de messages et alternatives réseau . . . 30
3.2.5 Intégration du chatbot avec système RAG . . . 30
3.2.6 WebSocket pour le suivi en temps réel . . . 31
3.2.7 Outils de développement . . . 31
3.3 Implémentation de la plateforme E-Santé Ibn Sina . . . 31
3.3.1 Mise en place du back-end . . . 31
3.3.2 Mise en place du front-end . . . 32
3.3.3 Mise en place de l'authentification . . . 33
3.3.4 Intégration du QR Code . . . 33
3.3.5 Intégration du suivi en temps réel . . . 33
3.4 Présentation des interfaces de l'application . . . 34
3.5 Tests fonctionnels . . . 39
3.5.1 Ordre des tests . . . 39
3.5.2 Tests de gestion des consultations . . . 39
3.5.3 Tests de sécurisation des ordonnances . . . 40
3.5.4 Tests de livraison et validation QR Code . . . 40

Conclusion générale et perspectives **41**
Bilan du projet . . . 41
Difficultés rencontrées . . . 41
Apports du projet . . . 41
Perspectives . . . 42

Références bibliographiques **43**

Dédicaces :

À nos familles, pour leur soutien indéfectible et leur patience tout au long de ce parcours.
À tous ceux qui luttent pour l'amélioration des conditions de santé et qui œuvrent au quotidien pour la dignité des patients.
À toutes les personnes qui ont contribué à ce projet et qui croient en un avenir où la technologie sert l'humain.
À l'équipe médicale et administrative du Centre Hospitalier Universitaire Ibn Sina pour leur engagement et leur professionnalisme.

L'équipe E-Santé Ibn Sina

Remerciements :

Nous remercions chaleureusement notre encadrant, Monsieur Yassine Sembati, Chef de Service de Management des Systèmes d'Information au CHU Ibn Sina, pour son accompagnement, ses précieux conseils et sa disponibilité tout au long de ce projet. Il nous a également mis en contact avec des ingénieurs DevOps et réseau de l'établissement, dont l'expertise et les conseils techniques nous ont été précieux pour la réalisation de la plateforme.

Nous remercions également l'équipe pédagogique de l'EMSI ainsi que toute l'équipe du Centre Hospitalier Universitaire Ibn Sina qui a contribué, de près ou de loin, à la réalisation de ce travail.

Un merci particulier au corps médical et administratif du CHU Ibn Sina pour leur collaboration et leur expertise qui ont permis d'orienter ce projet vers des solutions concrètes et adaptées aux besoins réels du terrain.

L’équipe E-Santé Ibn Sina

Résumé :

La plateforme **E-Santé Ibn Sina** est une solution web intelligente et centralisée de gestion des soins au sein du Centre Hospitalier Universitaire Ibn Sina. Elle met en relation patients, médecins, secrétaires, pharmaciens, livreurs et administrateurs pour optimiser le parcours de soin, du symptôme à la délivrance du traitement.

La plateforme intègre des modules avancés d'intelligence artificielle (AMIE, MIRA, MedCTM) pour l'analyse des symptômes, l'aide à la décision clinique et l'interprétation des images médicales. Elle propose également des fonctionnalités de géolocalisation, de suivi en temps réel des livraisons, de validation par QR Code, de messagerie instantanée et de paiement en ligne sécurisé.

Ce rapport présente l'analyse, la conception, l'implémentation et les tests de la solution, en détaillant les spécifications techniques, l'architecture du système et les interfaces développées pour chaque acteur.

Mots-clés : e-santé, parcours de soin, intelligence artificielle, FastAPI, React, RAG, blockchain, QR Code, CHU Ibn Sina.

Abstract :

The **E-Santé Ibn Sina** platform is an intelligent and centralized web solution for healthcare management within the Ibn Sina University Hospital Center. It connects patients, doctors, secretaries, pharmacists, delivery drivers and administrators to optimize the care pathway, from symptom to treatment delivery.

The platform integrates advanced artificial intelligence modules (AMIE, MIRA, MedCTM) for symptom analysis, clinical decision support and medical image interpretation. It also offers geolocation features, real-time delivery tracking, QR Code validation, instant messaging and secure online payment.

This report presents the analysis, design, implementation and testing of the solution, detailing the technical specifications, system architecture and interfaces developed for each stakeholder.

Keywords : e-health, care pathway, artificial intelligence, FastAPI, React, RAG, blockchain, QR Code, Ibn Sina University Hospital.

Chapitre 1

Contexte général du projet :

Introduction :

Face aux défis croissants du système de santé marocain et aux difficultés de coordination entre les différents acteurs du parcours de soin, le projet **E-Santé Ibn Sina** propose une solution web centralisée pour gérer l'ensemble des processus médicaux et administratifs au sein du CHU Ibn Sina.

Ce chapitre présente le contexte, la problématique, les objectifs et la solution proposée pour répondre aux enjeux de la transformation numérique du secteur de la santé.

1.1 Présentation générale du projet :

1.1.1 Contexte du projet :

Le Centre Hospitalier Universitaire Ibn Sina, établissement de référence au Maroc, fait face à une augmentation constante de la demande de soins et à une complexification des parcours patients. La digitalisation des processus de santé est devenue une nécessité pour améliorer la qualité des soins, réduire les délais de prise en charge et optimiser les ressources disponibles.

E-Santé Ibn Sina est une plateforme web intelligente qui facilite la gestion centralisée des soins en mettant en relation l'ensemble des acteurs du système de santé : patients, médecins, secrétaires, pharmaciens, livreurs et administrateurs.

1.1.2 Problématique :

Comment concevoir une plateforme centralisée, sécurisée et intelligente pour la gestion du parcours de soin au sein du CHU Ibn Sina, assurant le suivi des consultations, des prescriptions et des livraisons, tout en garantissant la coordination entre les acteurs ?

Les difficultés identifiées sont :

- **Absence d'un espace unique** regroupant tous les acteurs du système de santé.
- **Manque de traçabilité** des consultations, des prescriptions et des délivrances.
- **Difficulté de coordination** et de communication entre les intervenants.
- **Délais de prise en charge** trop longs pour les patients.

- **Gestion manuelle** des rendez-vous et des urgences.
- **Absence d'outils d'aide à la décision** basés sur l'intelligence artificielle.

1.1.3 Objectifs du projet :

E-Santé Ibn Sina vise à :

- **Centraliser et sécuriser** la gestion des utilisateurs et des données médicales.
- **Optimiser le parcours de soin** du symptôme à la délivrance du traitement.
- **Réduire les délais de prise en charge** via un triage intelligent.
- **Sécuriser et tracer** les ordonnances et les données médicales.
- **Faciliter la communication** entre les acteurs.
- **Intégrer des services d'intelligence artificielle** (AMIE, MIRA, MedCTM).
- **Assurer l'interopérabilité** avec d'autres systèmes de santé (FHIR).

1.1.4 Solution proposée :

E-Santé Ibn Sina est une application web composée :

- D'un **frontend React/TypeScript** avec Vite pour l'interface utilisateur.
- D'un **backend FastAPI (Python)** exposant des API REST sécurisées.
- D'une **base de données PostgreSQL** pour le stockage des données.
- De **microservices d'IA locaux** (Ollama/LLaMA) pour la confidentialité des données .
- De **Redis** pour le cache et la gestion des sessions.
- De **Docker** pour la conteneurisation et l'orchestration.

1.2 Étude de l'existant :

1.2.1 Analyse des solutions existantes :

Des solutions comme **Doctolib** (prise de rendez-vous) ou **MédiSafe** (gestion des ordonnances) couvrent des aspects spécifiques du parcours de soin, mais aucune ne propose une gestion intégrée de toute la chaîne : du symptôme à la délivrance du traitement.

Les établissements de santé utilisent encore souvent des outils disparates (tableurs, appels téléphoniques, formulaires papier), limitant la traçabilité et la coordination.

1.2.2 Limites des solutions actuelles :

- **Absence d'intégration** entre les différentes étapes du parcours de soin.
- **Pas de triage intelligent** pour orienter les patients vers la bonne ressource.
- **Pas de suivi en temps réel** des livraisons de médicaments.
- **Pas d'assistant intelligent** pour aider à la décision clinique.
- **Sécurité insuffisante** des prescriptions médicales.

1.2.3 Valeur ajoutée d'E-Santé Ibn Sina :

- **Centralisation** de tous les acteurs sur une même plateforme.
- **Traçabilité complète** des consultations, prescriptions et délivrances.
- **Triage intelligent** via IA (AMIE, MIRA, MedCTM).
- **Sécurisation des ordonnances** par blockchain et QR Code.
- **Suivi en temps réel** des livraisons de médicaments.
- **Interopérabilité** via standard FHIR.
- **IA locale** garantissant la confidentialité des données médicales.

Chapitre 2

Analyse et conception de la plateforme E-Santé Ibn Sina :

Introduction :

Ce chapitre présente l'analyse des besoins, les acteurs, les fonctionnalités principales, ainsi que les choix de conception de la plateforme E-Santé Ibn Sina. Une conception rigoureuse est essentielle pour garantir la cohérence du système et faciliter son développement.

2.1 Analyse des besoins :

2.1.1 Périmètre du projet :

Le périmètre d'E-Santé Ibn Sina couvre les fonctionnalités essentielles liées à la gestion du parcours de soin, des utilisateurs et des services associés. Il inclut la gestion des comptes, des consultations, des prescriptions, des missions de livraison, des rôles, les services de communication, de géolocalisation et d'assistance intelligente.

2.1.2 Acteurs de la plateforme :

La plateforme E-Santé Ibn Sina implique six acteurs principaux :

- **Patient** : gestion du parcours de soin, saisie des symptômes, prise de rendez-vous, paiement en ligne, suivi des livraisons.
- **Médecin** : consultations cliniques, diagnostics, prescriptions, utilisation de l'IA, visionneuse 3D anatomique.
- **Secrétaire** : validation des inscriptions, gestion des rendez-vous, envoi de rappels, transmission des ordonnances.
- **Pharmacien** : vérification des ordonnances, délivrance des médicaments, suivi des stocks.
- **Livreur** : gestion des missions de livraison, suivi GPS, validation par QR Code.
- **Administrateur** : supervision, validation des comptes, gestion des paramètres, audit des logs.

2.1.3 Diagrammes des cas d'utilisation:

Les diagrammes des cas d'utilisation présentent les interactions entre les six acteurs principaux de la plateforme E-Santé Ibn Sina et les fonctionnalités clés du système.

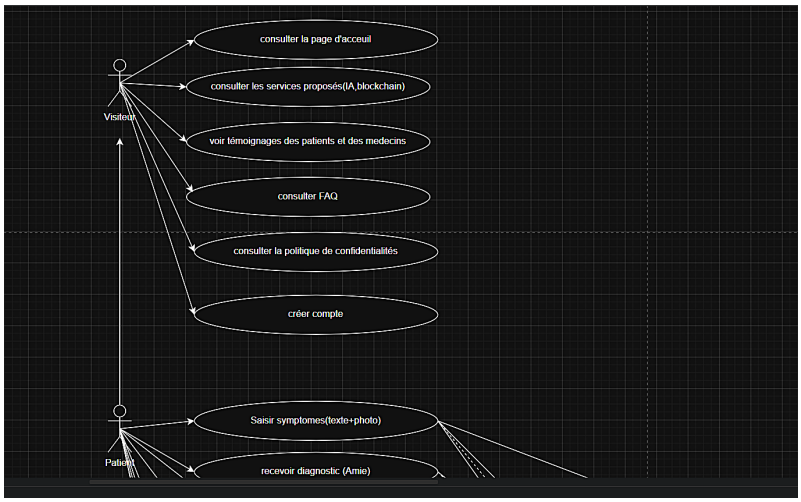

Figure 2.1: Diagramme des cas d'utilisation du visiteur

Le visiteur peut consulter les informations publiques sur les services de l'hôpital, s'inscrire en tant que patient, et visualiser la localisation des établissements de santé. Il existe une généralisation entre le visiteur et le patient, car le patient peut réaliser toutes les fonctionnalités d'un visiteur en plus des fonctionnalités spécifiques à son rôle.

Figure 2.2: Diagramme des cas d'utilisation du patient

Le patient s'authentifie sur la plateforme pour accéder à son espace personnel. Il peut saisir ses symptômes (texte et photo), recevoir une aide au diagnostic via les modules IA (AMIE, MIRA, MedCTM), prendre rendez-vous avec un médecin, consulter son historique médical, télécharger ses ordonnances sécurisées et effectuer des paiements en ligne.

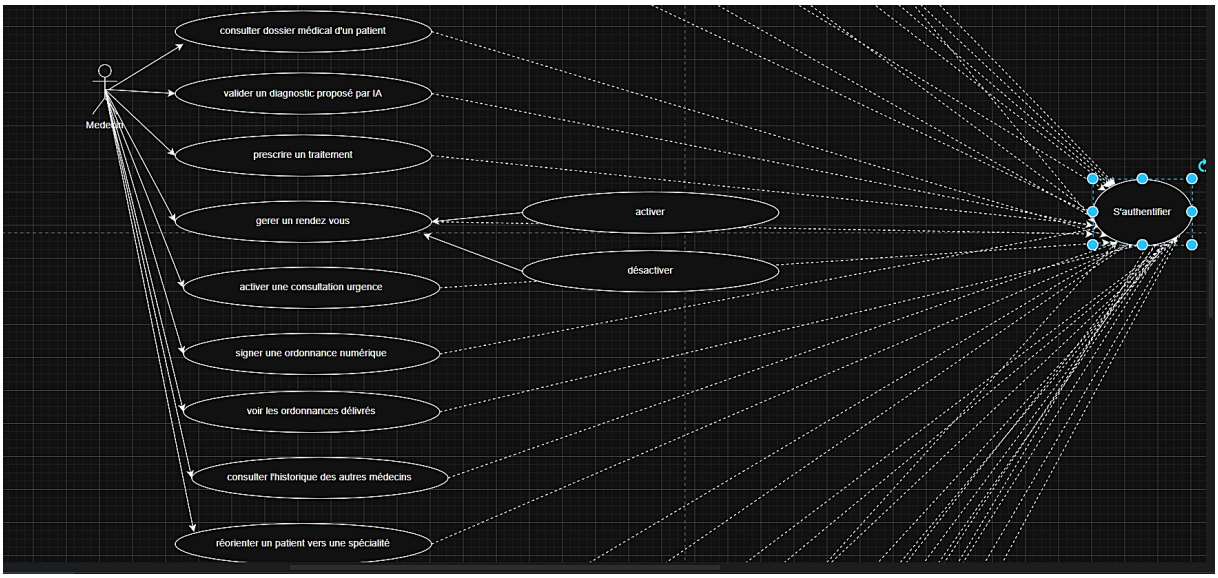

Figure 2.3: Diagramme des cas d'utilisation du médecin

Le médecin s'authentifie pour accéder à son espace de travail clinique. Il peut consulter les dossiers patients, valider ou ajuster les diagnostics proposés par l'IA, prescrire des traitements, signer numériquement les ordonnances, déclencher des consultations d'urgence, réorienter les patients vers des spécialistes et utiliser la visionneuse 3D anatomique pour examiner le corps humain.

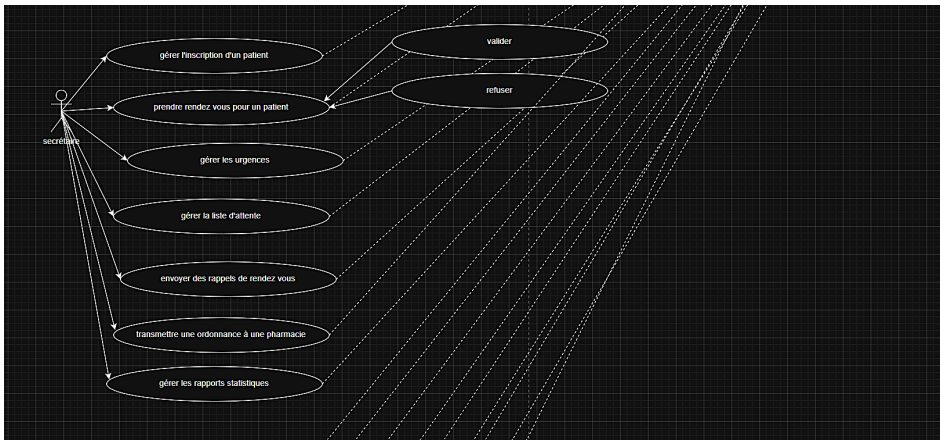

Figure 2.4: Diagramme des cas d'utilisation du secrétaire

Le secrétaire s'authentifie pour gérer les aspects administratifs de la plateforme. Il peut valider ou refuser les inscriptions des patients, gérer les rendez-vous et les urgences, générer des statistiques d'activité et auditer les accès aux dossiers médicaux.

Figure 2.5: Diagramme des cas d'utilisation du pharmacien

Le pharmacien s'authentifie pour accéder à son espace professionnel. Il peut vérifier l'authenticité des ordonnances via blockchain, délivrer les médicaments, marquer les ordonnances comme "délivrées", recevoir des alertes d'interaction médicamenteuse et consulter l'historique pharmaceutique des patients.

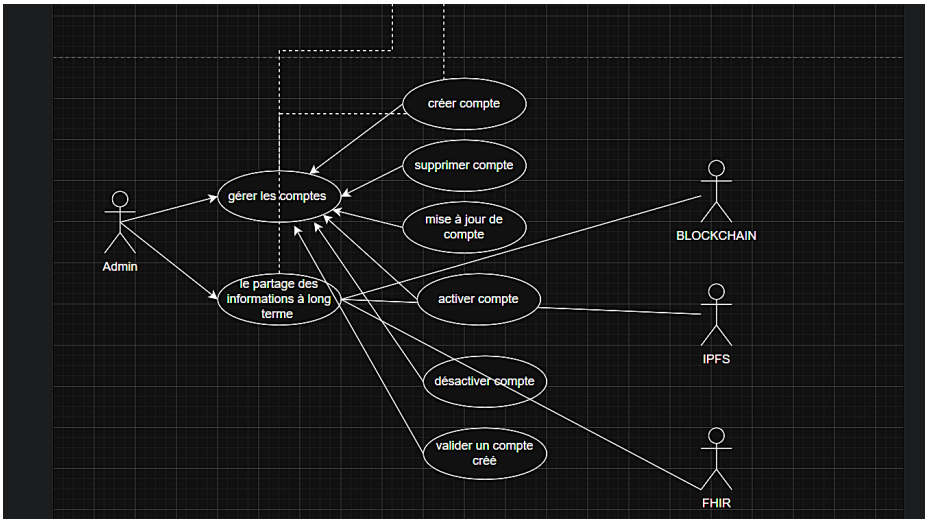

Figure 2.6: Diagramme des cas d'utilisation de l'administrateur

L'administrateur s'authentifie pour superviser l'ensemble de la plateforme. Il peut valider les inscriptions des médecins et pharmaciens, activer ou désactiver les API externes, configurer les paramètres globaux, gérer les sauvegardes et consulter les logs d'audit.

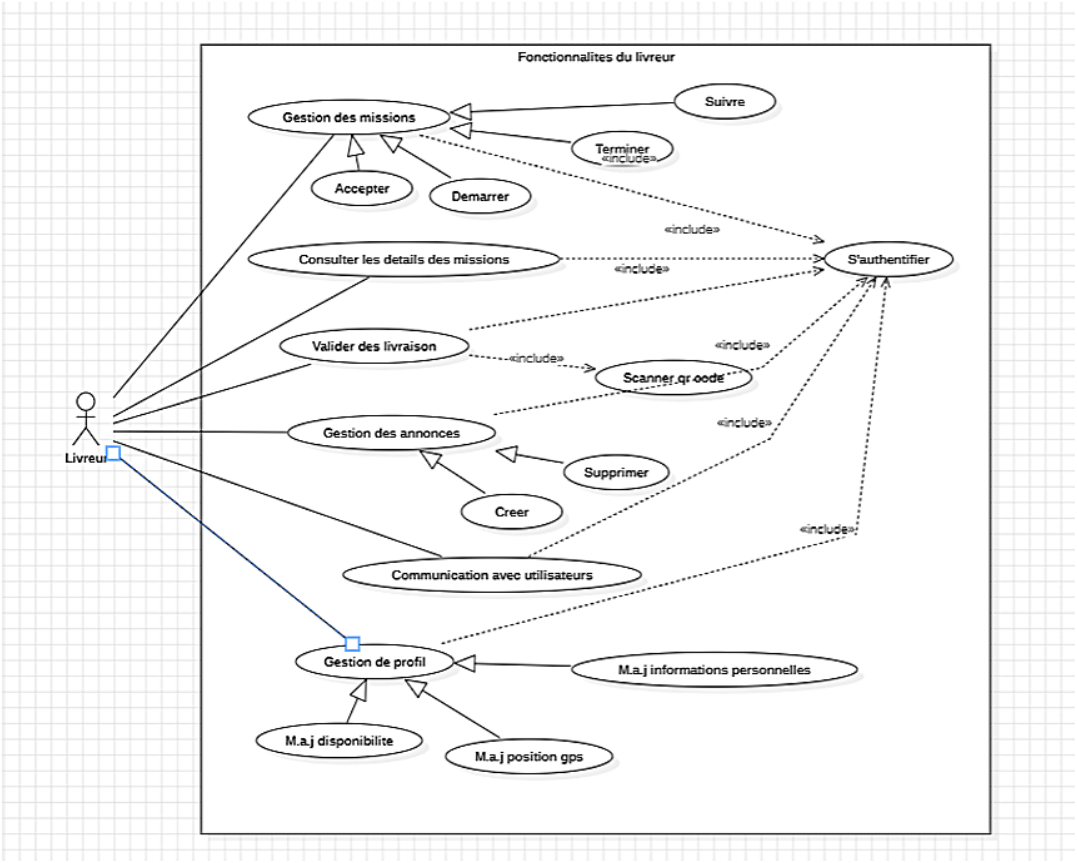

Figure 2.7: Diagramme des cas d'utilisation du livreur

Le livreur s'authentifie pour gérer ses missions de livraison. Il peut consulter les missions disponibles, communiquer avec les autres profils (pharmacien, patient), gérer ses annonces, valider les livraisons via QR Code et gérer son profil.

2.1.4 Diagrammes de séquence :

Les diagrammes de séquence ci-dessous illustrent les interactions entre les différents composants du système pour les fonctionnalités clés, organisées par acteur.

Acteur : Patient

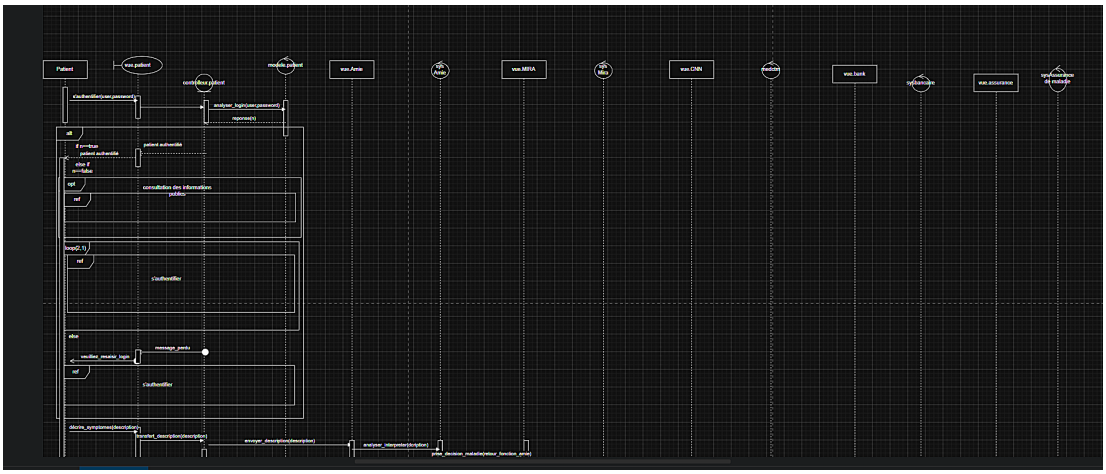

Figure 2.8: Diagramme de séquence - Authentification du patient avec gestion des erreurs

Lors de l'authentification, le patient saisit ses identifiants (email et mot de passe) via l'interface de connexion. Le système vérifie ces informations auprès de la base de données. Si les identifiants sont corrects, un token JWT est généré et retourné au patient. Ce

dernier accède alors à son tableau de bord personnalisé. En cas d'erreur de connexion, le système affiche un message d'erreur explicite et propose à l'utilisateur de réessayer après correction. Après 5 tentatives échouées, le compte est temporairement verrouillé pendant 15 minutes pour prévenir les attaques par force brute. Un lien "Mot de passe oublié" permet d'initier une réinitialisation par email. En cas de perte de connexion réseau, le système met en file d'attente les requêtes et tente automatiquement de les retransmettre une fois la connexion rétablie.

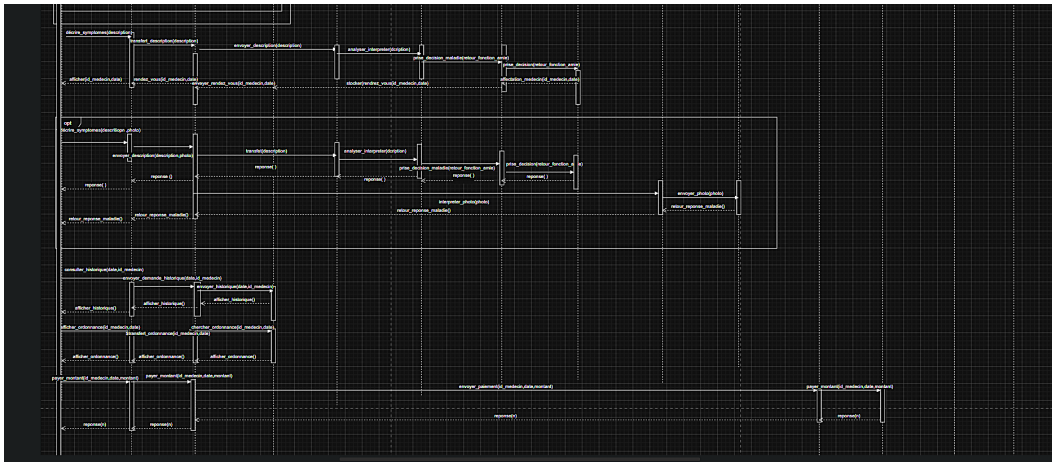

Figure 2.9: Diagramme de séquence - Saisie des symptômes et diagnostic IA

Le patient envoie une description de ses symptômes (texte et/ou photo). Le système transmet ces données aux modules IA (AMIE pour l'interprétation textuelle, MedCTM pour l'analyse des images). Les résultats sont agrégés et présentés au patient sous forme de diagnostic intelligent. Le système affiche également le niveau d'urgence via MIRA. Le patient peut consulter l'historique des ses maladies en filtrant par date ou par identifiant de médecin. Parmi les options offertes, le patient peut effectuer un paiement en ligne en appelant des services de paiement réels, en spécifiant la date, le médecin concerné et le montant à payer.

Figure 2.10: Diagramme de séquence - Demande de remboursement

Le patient peut envoyer une demande de remboursement à une assurance maladie en saisissant toutes les informations nécessaires. Cette fonctionnalité comporte des alternatives comme en cas de frais non couverts par l'assurance, où le patient doit payer lui-même.

Acteur : Médecin

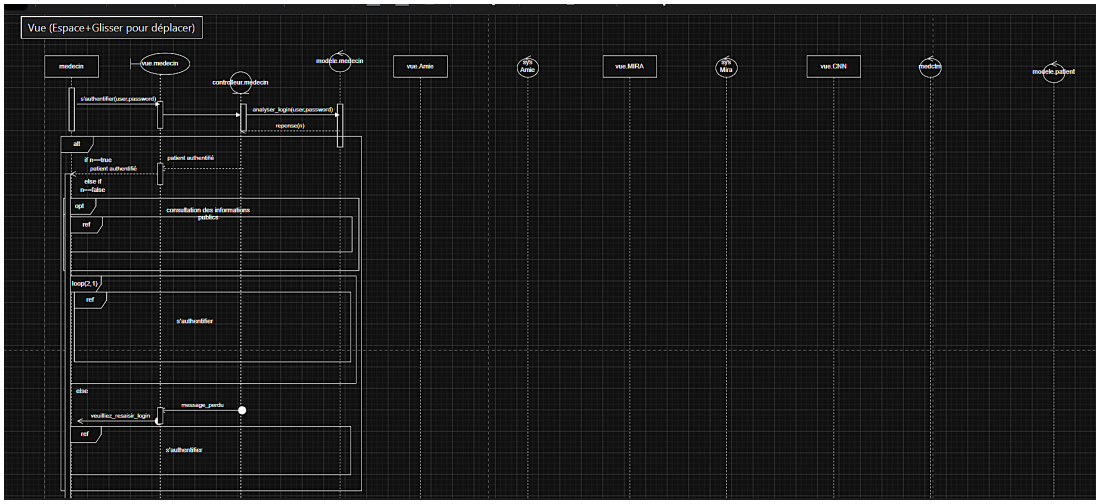

Figure 2.11: Diagramme de séquence - Authentification du médecin

Le médecin s'authentifie sur la plateforme. En cas d'utilisateur non authentifié, il est considéré comme un visiteur ne pouvant que consulter des informations publiques. Le système gère également les cas de perte de messages de réponse venant des modèles IA.

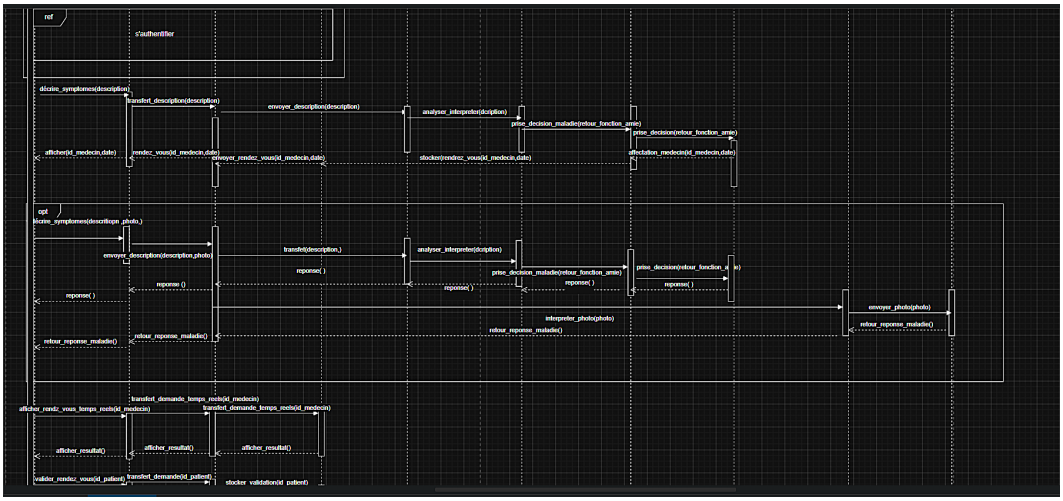

Figure 2.12: Diagramme de séquence - Utilisation de l'IA et gestion des rendez-vous

Le médecin peut utiliser l'intelligence artificielle pour préciser son diagnostic en se basant sur les descriptions et les photos de radiologie, de gonflements ou d'inflammations. Il peut également consulter les dates des rendez-vous qui lui sont affectés et préciser un rendez-vous avec un patient spécifique en saisissant son identifiant.

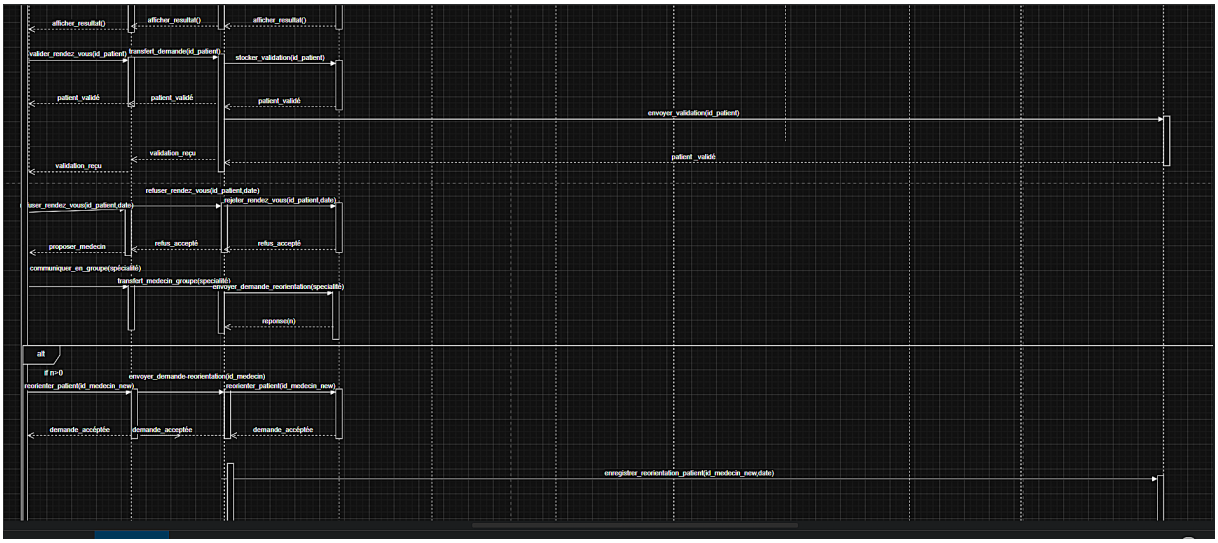

Figure 2.13: Diagramme de séquence - Réorientation vers un spécialiste

Si le médecin constate que la maladie du patient relève d'une autre spécialité, il peut, via des groupes entre médecins, proposer une réorientation du patient vers le médecin acceptant, en se basant sur le diagnostic et sur l'emploi du temps de l'autre médecin.

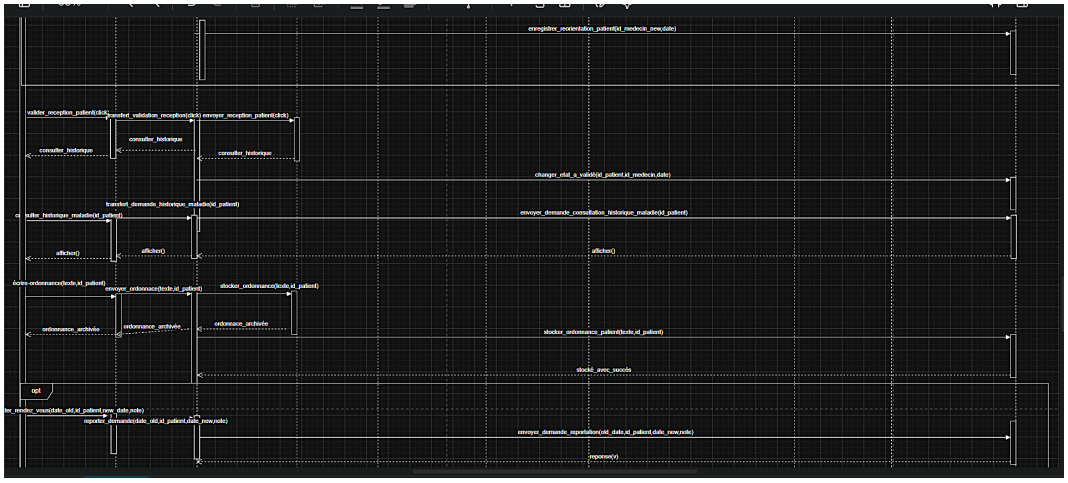

Figure 2.14: Diagramme de séquence - Validation et création d'ordonnance

Le médecin peut valider ou refuser la réception d'un patient. Il peut afficher l'historique des maladies du patient via son IPP unique et créer des ordonnances contenant une signature validant la crédibilité de la source. En cas de refus d'un patient, il peut organiser un autre rendez-vous avec une date de son choix, validée par le secrétaire.

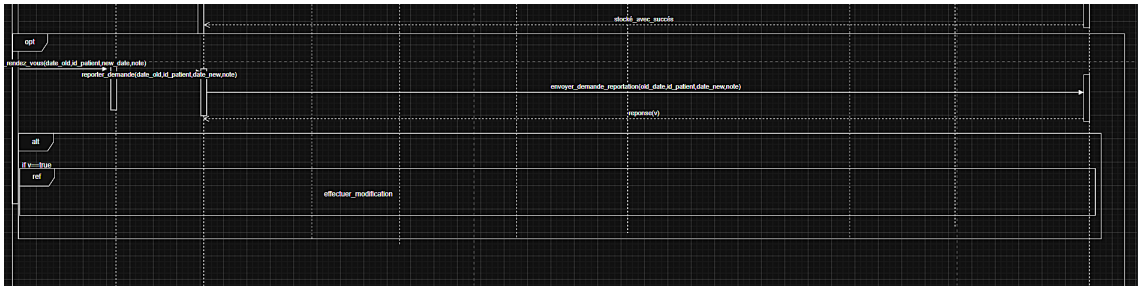

Figure 2.15: Diagramme de séquence - Modification du diagnostic et de l'ordonnance

Le médecin peut à tout moment modifier le diagnostic du patient ainsi que son ordonnance, avec traçabilité des modifications.

Acteur : Secrétaire

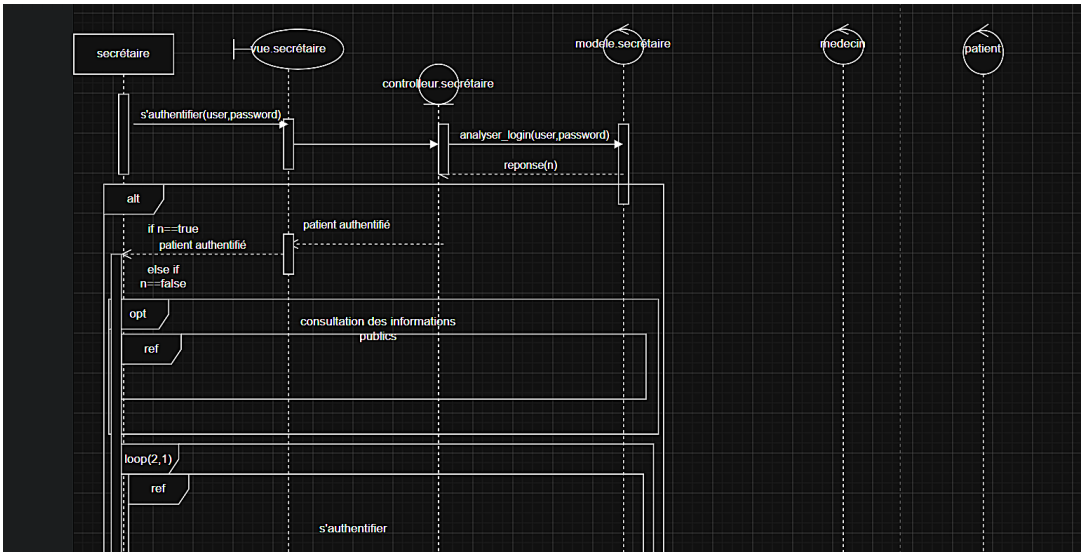

Figure 2.16: Diagramme de séquence - Authentification du secrétaire

Le secrétaire s'authentifie sur la plateforme. En cas de perte de message de réponse venant des modèles, le système gère la situation avec des alternatives appropriées.

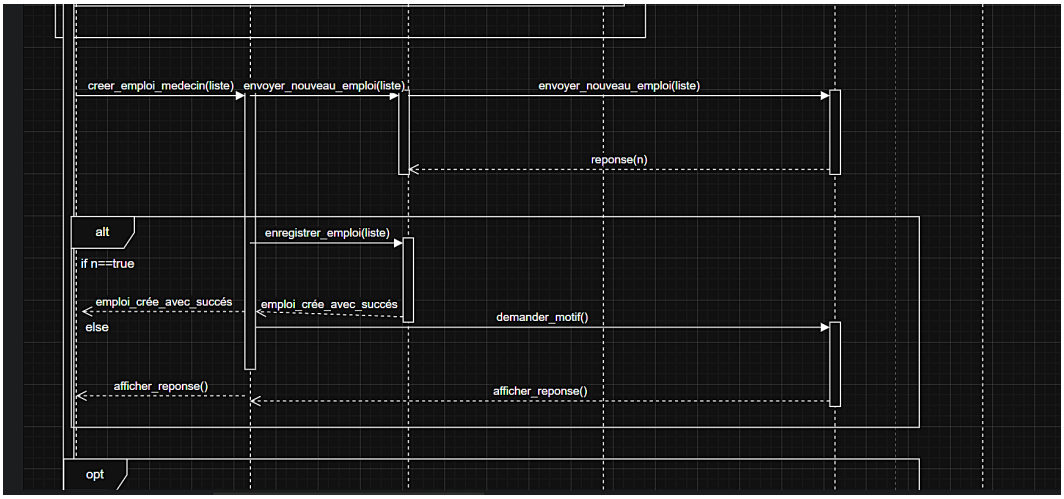

Figure 2.17: Diagramme de séquence - Création de l'emploi du temps du médecin

Le secrétaire peut créer l'emploi du temps du médecin en définissant les plages horaires de consultation et de disponibilité.

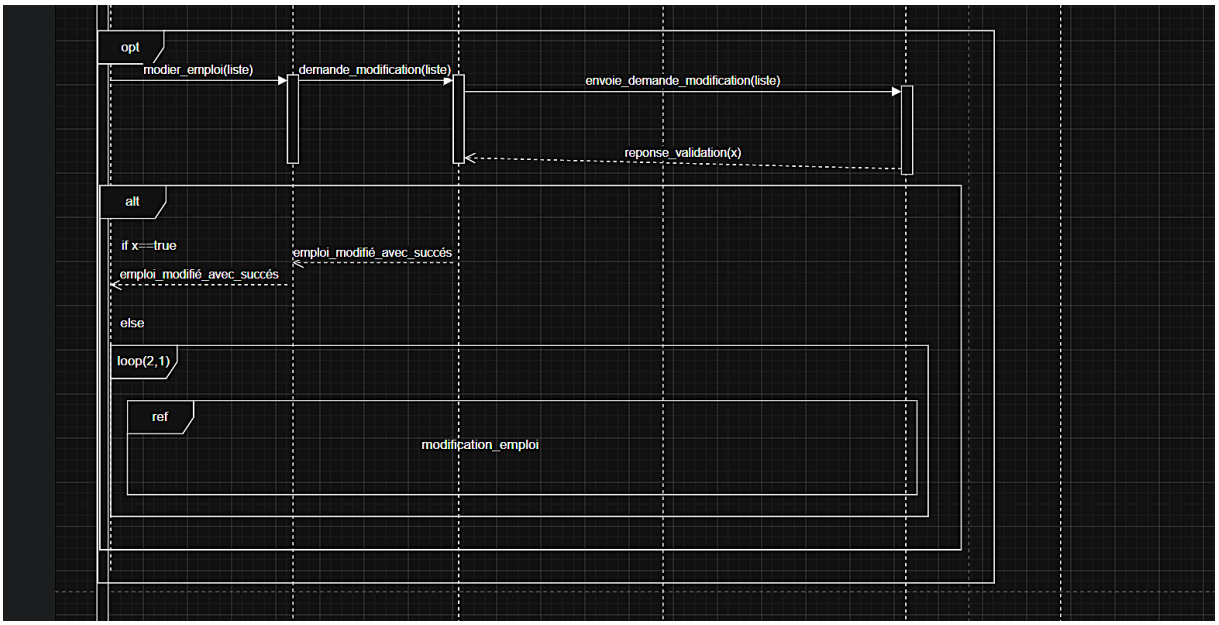

Figure 2.18: Diagramme de séquence - Modification de l'emploi du temps

Le secrétaire peut modifier l'emploi du temps du médecin. En cas de modifications non valides, une boucle de 2 autres tentatives est appelée.

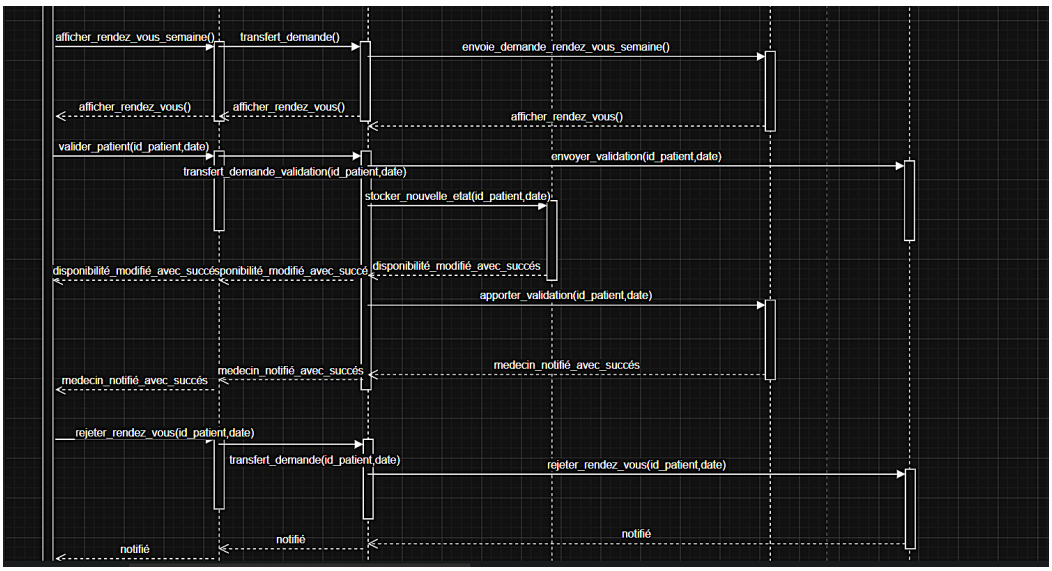

Figure 2.19: Diagramme de séquence - Validation des rendez-vous

Le secrétaire peut afficher l'emploi du temps du médecin et valider le rendez-vous d'un patient en précisant la date. Le médecin et le patient sont notifiés. Il peut également rejeter une demande de rendez-vous en précisant l'IPP du patient et la date.

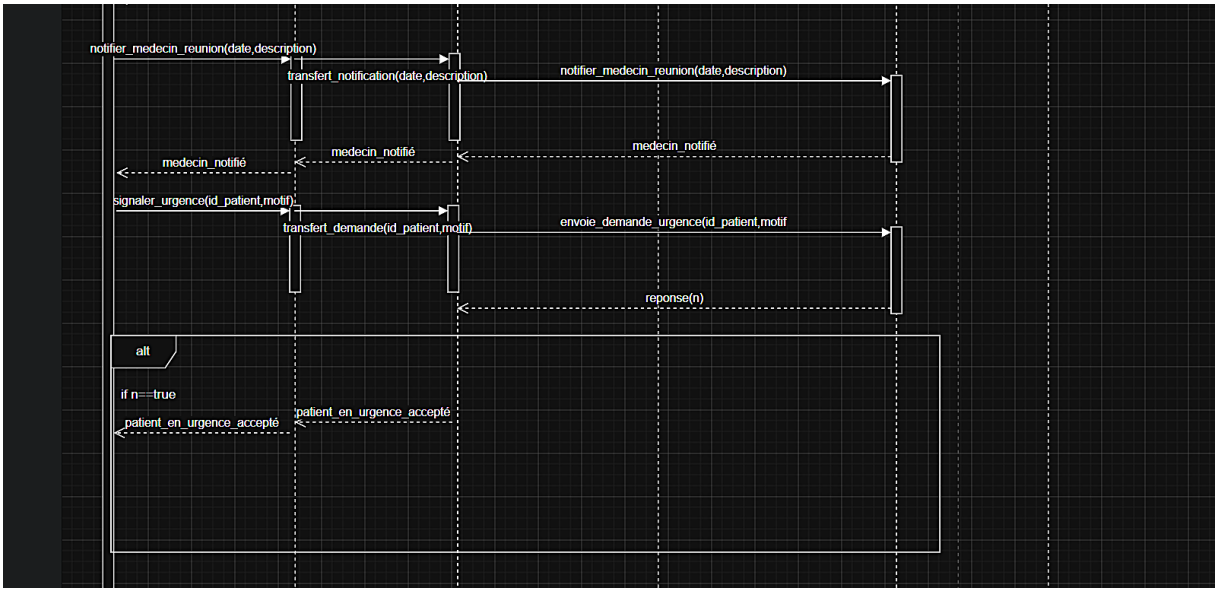

Figure 2.20: Diagramme de séquence - Notifications et urgences

Le secrétaire est responsable de notifier le médecin d'une réunion organisée par l'hôpital et de signaler une urgence en précisant l'IPP du patient et le motif de l'urgence, avec gestion des alternatives.

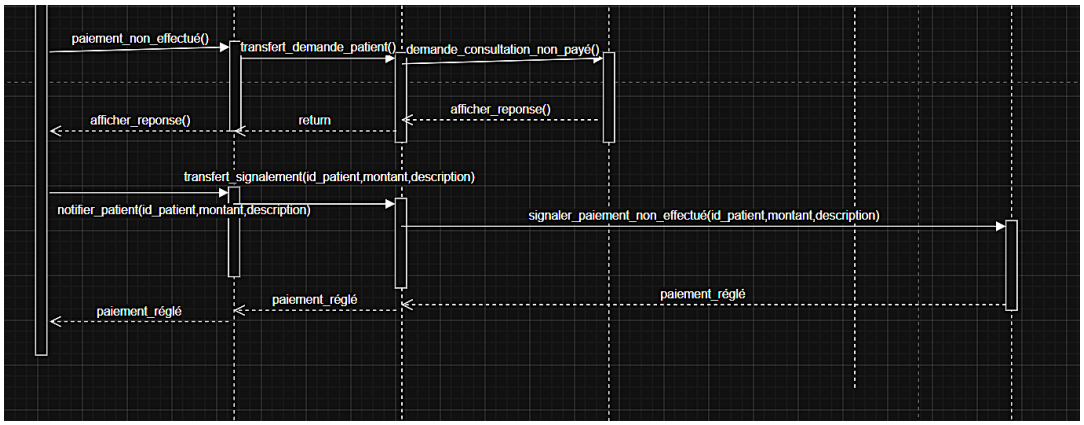

Figure 2.21: Diagramme de séquence - Validation des paiements

Le secrétaire peut valider le paiement effectué par le patient et signaler au patient les rendez-vous non payés.

Acteur : Pharmacien

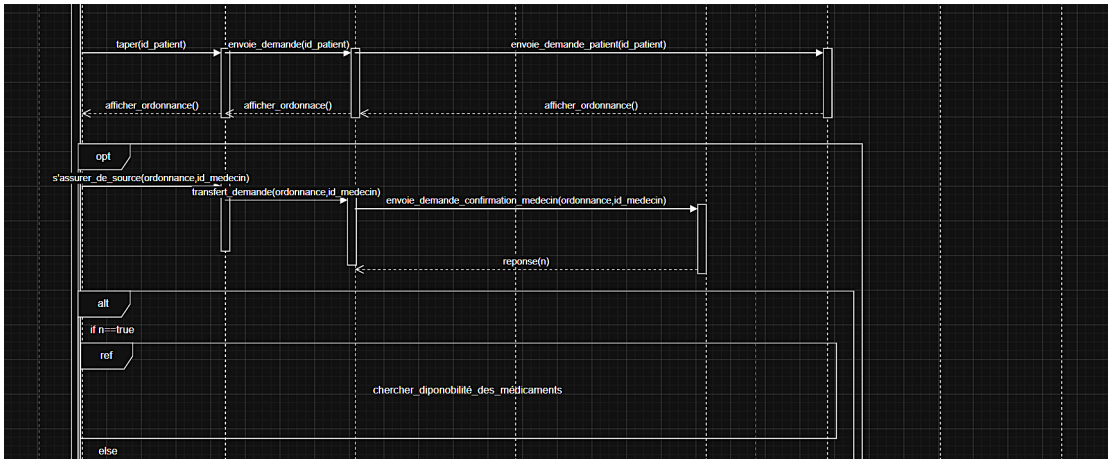

Figure 2.22: Diagramme de séquence - Consultation de l'historique et vérification de l'ordonnance

Via le principe de standard FHIR et de blockchain, le pharmacien peut voir l'historique des maladies du patient via son IPP et s'assurer de la validité de son ordonnance, qui est hachée une fois écrite par le médecin et stockée via le principe IPFS. Le pharmacien peut indiquer au patient la disponibilité des médicaments.

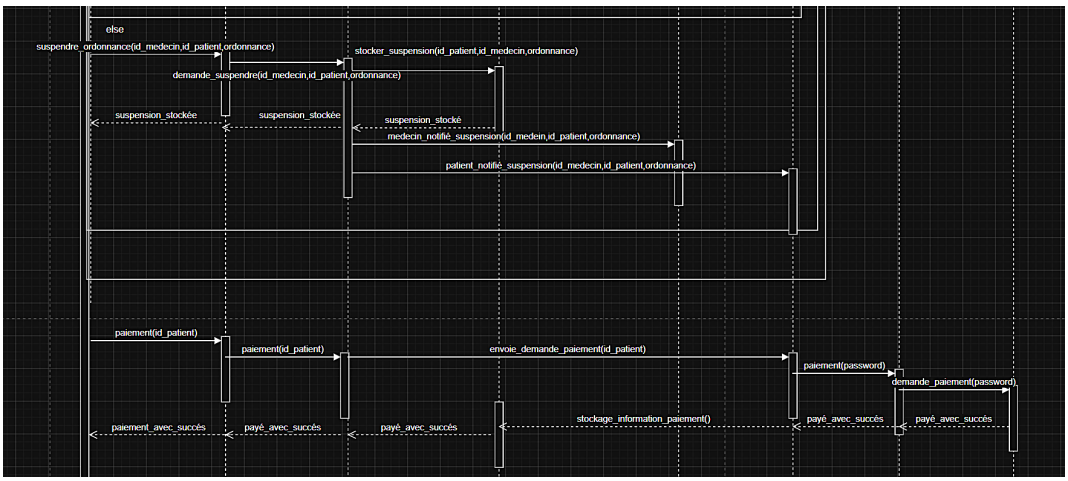

Figure 2.23: Diagramme de séquence - Suspension et validation du paiement

Le pharmacien peut suspendre une ordonnance et valider le paiement du patient, soit en présentiel, soit en ligne.

Acteur : Administrateur

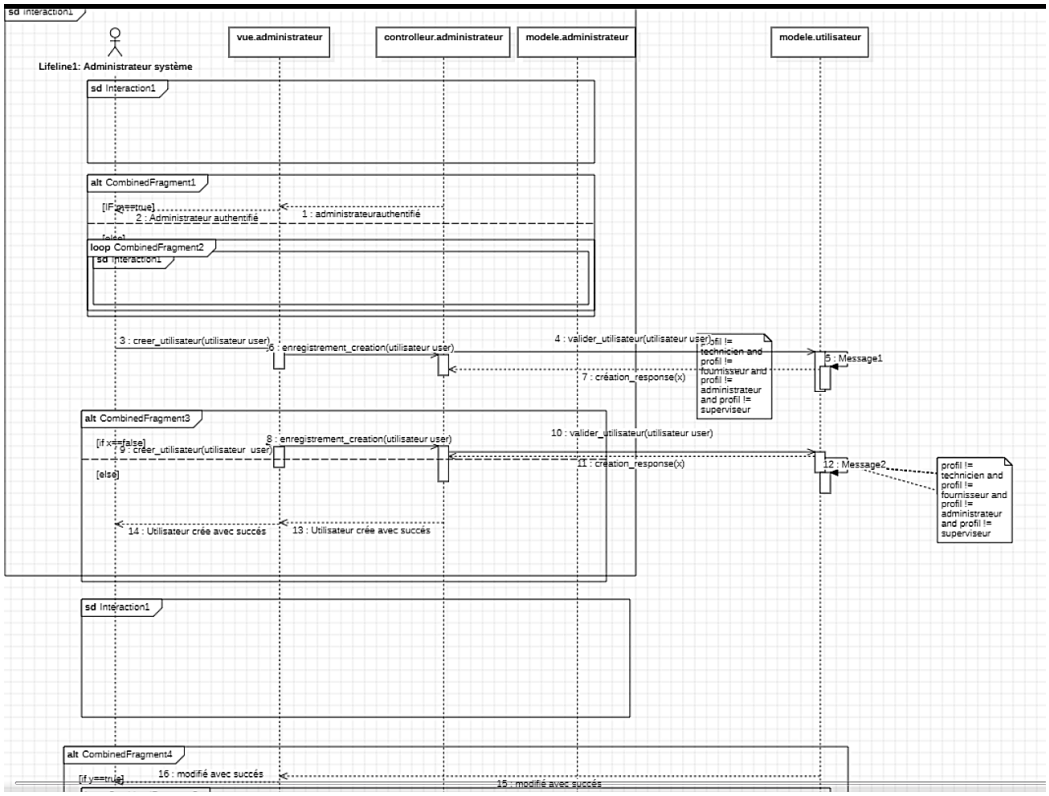

Figure 2.24: Diagramme de séquence - Authentification de l'administrateur

L'administrateur s'authentifie sur la plateforme. Le premier bloc de référence chez l'admin indique tout le processus d'authentification avec des alternatives et des accès publics en cas d'authentification non réalisée.

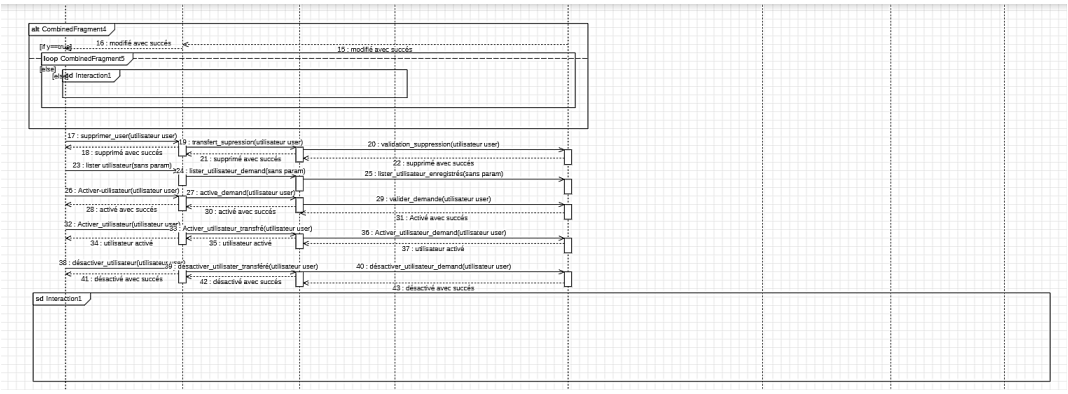

Figure 2.25: Diagramme de séquence - Gestion des comptes

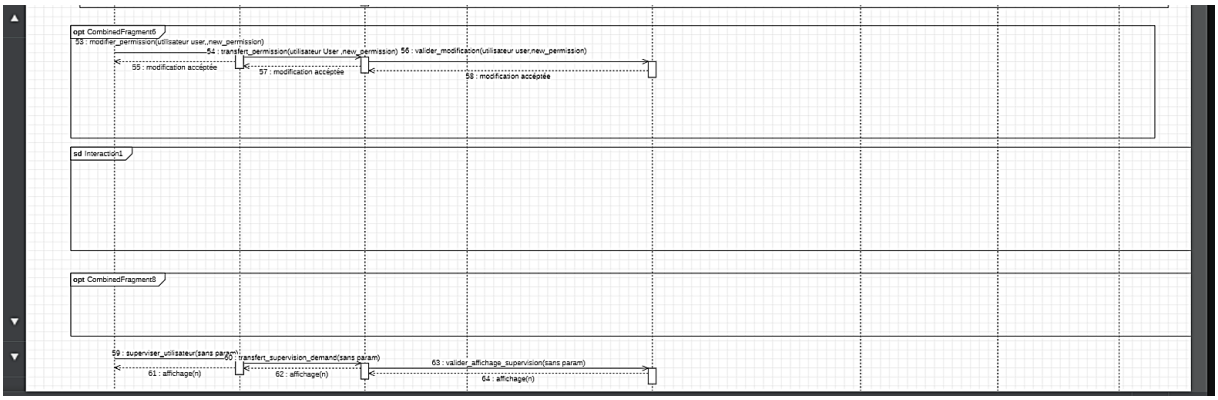

Figure 2.26: Diagramme de séquence - Gestion des comptes (suite)

L'administrateur peut activer ou désactiver un compte, valider ou rejeter la création d'un compte, et gérer l'ensemble des comptes (création, mise à jour, lecture, suppression) avec des références et des alternatives.

Acteur : Livreur

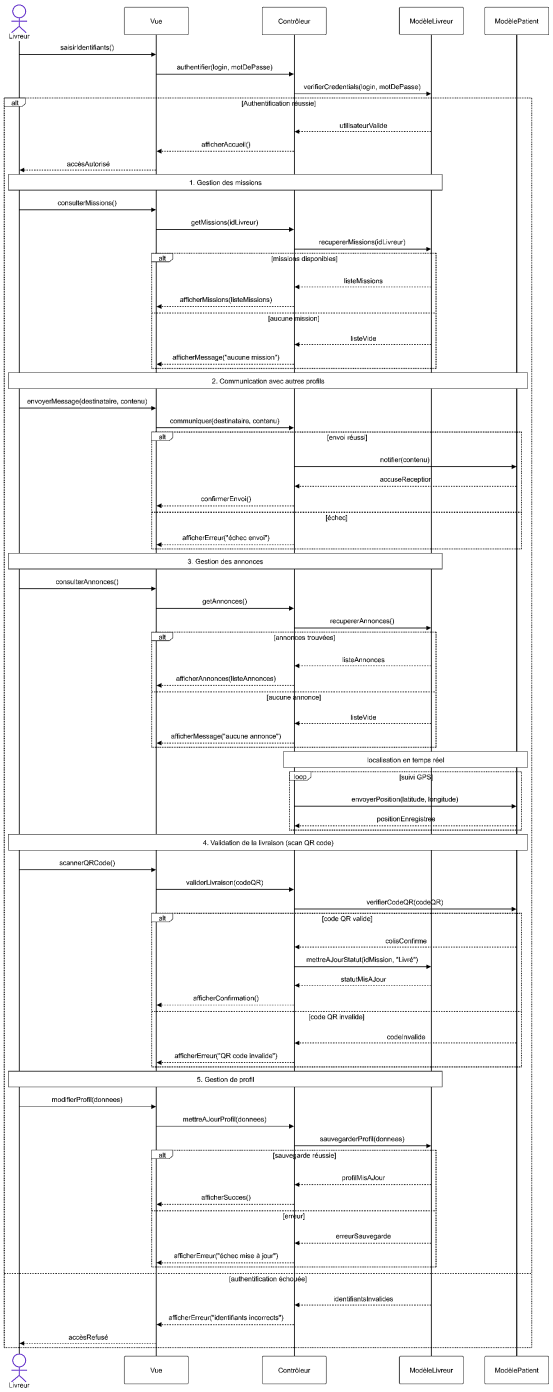

Figure 2.27: Diagramme de séquence - Gestion des missions du livreur

Le livreur s'authentifie d'abord auprès du système ; si les identifiants sont incorrects, l'accès lui est refusé. Une fois connecté, il peut consulter ses missions, qui lui sont affichées par le Contrôleur après récupération auprès du modèle. Il peut également communiquer avec les autres profils (pharmacien, patient), le Contrôleur se chargeant de transmettre le message via le modèle Patient et de confirmer sa bonne réception.

Le livreur consulte aussi les annonces disponibles, affichées après récupération des données. Pendant sa tournée, sa position (latitude, longitude) est envoyée régulièrement au modèle Patient pour permettre un suivi en temps réel de la livraison. Pour valider la

livraison, le livreur scanne le QR code du colis ; le Contrôleur vérifie ce code auprès du modèle Patient, puis met à jour le statut de la mission si le code est valide, ou affiche une erreur dans le cas contraire. Enfin, le livreur peut modifier son profil, les nouvelles informations étant sauvegardées via le modèle. Chaque action est soit validée avec succès, soit rejetée avec un message d'erreur.

2.1.5 Diagramme de classes :

Le diagramme de classes présente ci-dessous décrit la structure statique du système, en montrant les principales entités et leurs relations.

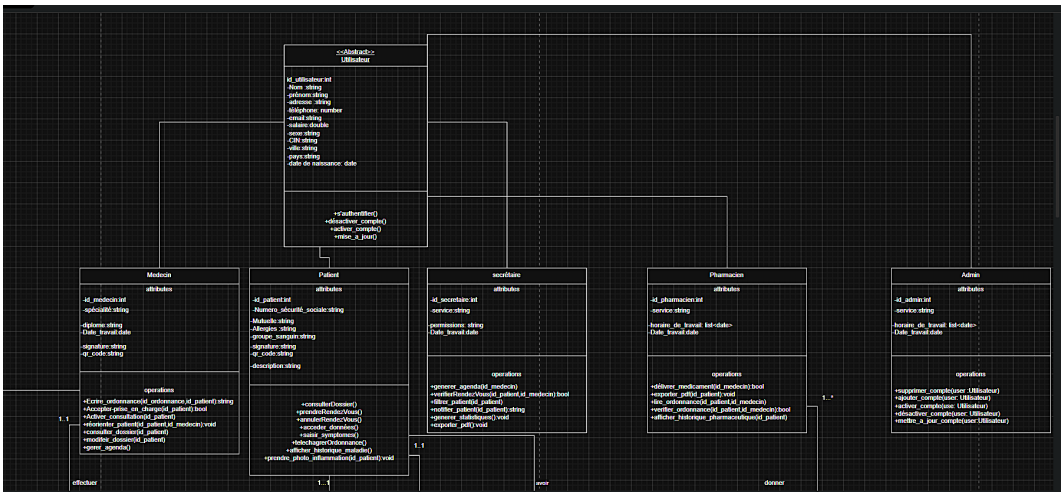

Figure 2.28: Diagramme de classes - Acteurs et entités principales

La classe abstraite `User` définit les attributs et méthodes communs à tous les acteurs (id, email, password, nom, prénom, role, dateCreation, enabled). Les classes `Patient`, `Doctor`, `Secretary`, `Pharmacist`, `DeliveryMan` et `Admin` héritent de `User` et implémentent les méthodes spécifiques à chaque rôle.

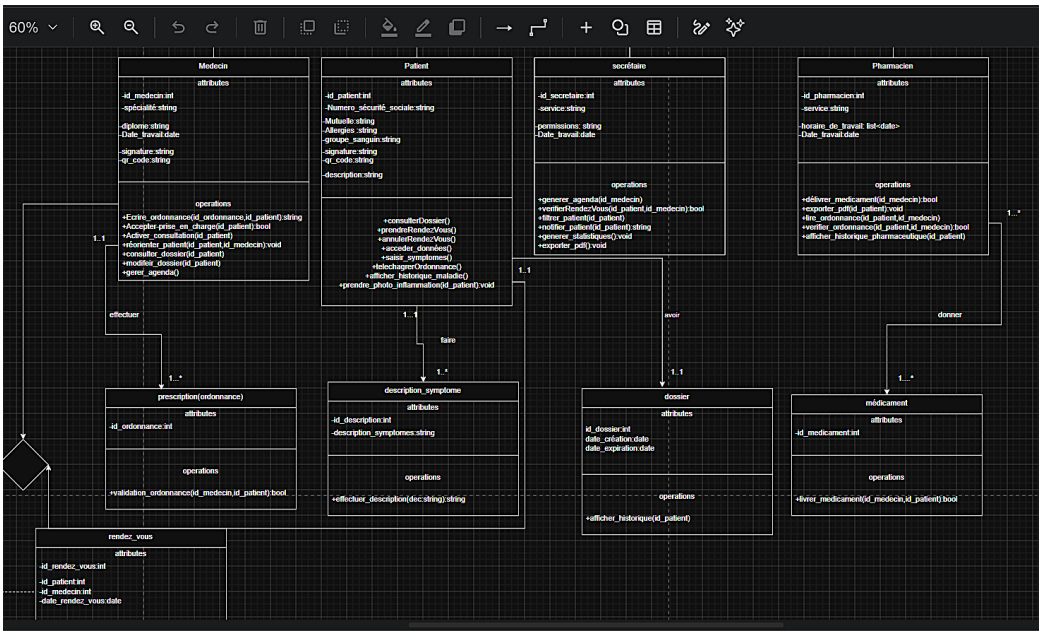

Figure 2.29: Diagramme de classes - Consultations, prescriptions et médicaments

Cette partie du diagramme montre les relations entre `Doctor` et `Patient` à travers l'entité `Appointment`. Un médecin peut effectuer une ou plusieurs prescriptions (`Prescription`) et un patient peut décrire ses symptômes, stockés dans son dossier médical (`MedicalRecord`). Le pharmacien (`Pharmacist`) peut délivrer des médicaments correspondant aux ordonnances.

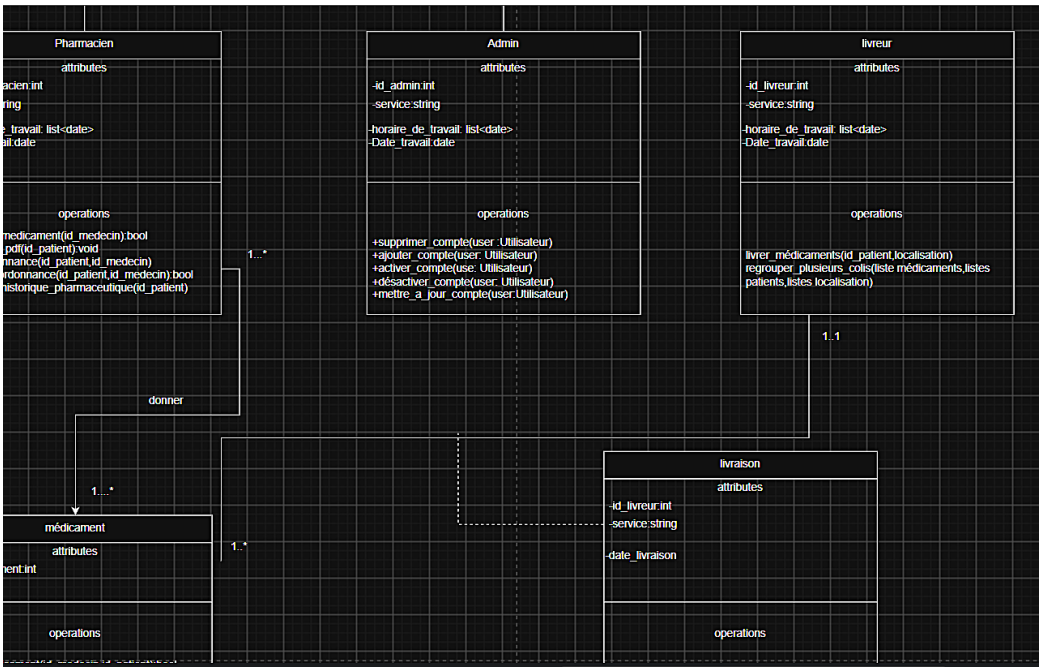

Figure 2.30: Diagramme de classes - Livraisons

Le livreur (`DeliveryMan`) peut être responsable de la livraison à un seul patient ou à plusieurs patients en même temps, sous réserve que les patients partagent les mêmes contraintes géographiques et la même date de livraison.

2.2 Conception de la base de données:

2.2.1 Modèle conceptuel:

Le modèle repose sur une table centrale `User` avec des profils spécialisés selon le rôle. Les principales entités et leurs relations sont :

Entité source	Relation	Entité cible
User	1-*	Profiles (Patient, Doctor, Secretary, Pharmacist, DeliveryMan, Admin)
Patient	1-*	Appointment
Doctor	1-*	Appointment
Doctor	1-*	Prescription
Patient	1-*	MedicalRecord
Pharmacist	1-*	Prescription
DeliveryMan	1-*	DeliveryOrder
Appointment	1-1	Prescription

Table 2.1: Principales relations de la base de données

2.2.2 Liste des tables de la base de données :

La base de données E-Santé Ibn Sina est composée des 33 tables suivantes, organisées par catégorie fonctionnelle :

Nom de la table	Description
1. Gestion des utilisateurs	
User	Table centrale des comptes utilisateurs (authentification, email, mot de passe, rôle, statut, localisation)
Patient	Profil spécifique aux patients (IPP, antécédents, groupe sanguin)
Doctor	Profil spécifique aux médecins (RPPS, spécialité, établissement)
Secretary	Profil spécifique aux secrétaires
Pharmacist	Profil spécifique aux pharmaciens (numéro d'ordre, établissement)
DeliveryMan	Profil spécifique aux livreurs (vérification, disponibilité, motorisation)
Admin	Profil spécifique aux administrateurs
2. Gestion des consultations et rendez-vous	
Appointment	Gestion des rendez-vous (date, statut, urgence)
ConsultationRequest	Demandes de consultation des patients
MedicalConsultation	Consultations médicales
Diagnosis	Diagnostics médicaux (code CIM-10)
PrescribedExam	Examens prescrits
MedicalRecord	Dossiers médicaux des patients
MedicalReport	Rapports médicaux
Schedule	Emplois du temps des médecins
AvailabilitySlot	Créneaux de disponibilité
3. Gestion des prescriptions	
Prescription	Prescriptions médicales (signature numérique, hash)
PrescriptionLine	Lignes de prescription
PrescriptionHistory	Historique des prescriptions
PrescriptionDelivery	Délivrance des prescriptions
DeliveryOrder	Commandes de livraison
DeliveryEvent	Événements de livraison
Treatment	Traitements
4. Intelligence Artificielle	
TriageAnalysis	Analyses de triage IA
MatchingScore	Scores de matching IA
MiraConversation	Conversations avec l'assistant MIRA
MiraMessage	Messages MIRA
AiRecommendation	Recommandations IA
AnalyticsReport	Rapports analytiques
AnalyticsAlert	Alertes analytiques
5. Communication et notifications	
Notification	Notifications envoyées aux utilisateurs

Suite de la table

Nom de la table	Description
News	Actualités

Chapitre 3

Réalisation et interfaces de l'application :

Introduction :

Ce chapitre présente l'environnement de développement, l'architecture technique, la structure du projet, les modules implémentés, les interfaces et les tests.

3.1 Environnement de développement :

3.1.1 Environnement matériel :

Le projet est développé sur un poste local capable d'exécuter simultanément :

- Un serveur backend FastAPI.
- Un serveur frontend Vite.
- Une base PostgreSQL.
- Un serveur Redis pour le cache.
- Un serveur Ollama pour l'IA locale.
- Docker pour l'orchestration des conteneurs.

3.1.2 Environnement logiciel :

L'environnement logiciel comprend :

- Python 3.12+.
- FastAPI 0.115.0.
- React 18.3.1 avec TypeScript.
- Vite 6.0.7.
- PostgreSQL 16.
- Redis 7.

- Ollama avec LLaMA 3.2.
- Docker & Docker Compose v2+.
- Swagger/OpenAPI pour consulter les endpoints.

3.2 Description des besoins techniques :

3.2.1 Architecture générale de l'application :

L'architecture est de type client-serveur 3-tier découplée et conteneurisée :

- Le navigateur charge l'application React SPA.
- React communique avec FastAPI via des API REST JSON asynchrones.
- FastAPI expose des API REST et des microservices d'IA locaux.
- PostgreSQL stocke les données relationnelles.
- Redis gère le cache et les sessions.
- Ollama exécute les modèles LLM localement (LLaMA 3.2).
- Docker orchestre les 5 conteneurs : `health_postgres`, `health_redis`, `health_backend`, `health_frontend`, `health_ollama`.

3.2.2 Technologies utilisées :

Couche	Technologies
Frontend	React 18.3.1, TypeScript 5.7.2, Vite 6.0.7, Tailwind CSS 3.4.17, React Router DOM 7.18.2, Zustand 5.0.3, TanStack React Query 5.62.16
3D & Visualisation	Three.js 0.185.1, GLTFLoader, OrbitControls, Raycasting, Leaflet 1.9.4, Recharts 3.9.2
Backend	Python 3.12+, FastAPI 0.115.0, Uvicorn 0.32.0, SQLAlchemy 2.0.36 (asyncio), Alembic 1.14.0, Pydantic 2.10.3
Base de données	PostgreSQL 16, asyncpg 0.30.0, Redis 7
Authentification	JWT (python-jose 3.3.0), Passlib 1.7.4 (Bcrypt), RBAC
Intelligence Artificielle	Ollama, LLaMA 3.2, Architecture RAG, Streaming UI
Sécurité	CORS, TrustedHost, Audit Logging, SHA-256, Blockchain, IPFS, FHIR
Tests	Pytest 8.3.4, pytest-asyncio 0.24.0, Vitest 4.1.11, Testing Library
Qualité	Ruff 0.8.4, Mypy 1.14.0, Pre-commit 4.0.1, ESLint 9.17.0
Infrastructure	Docker, Docker Compose v2+, Nginx, Healthchecks

3.2.3 Gestion des erreurs d'authentification et alternatives:

La plateforme E-Santé Ibn Sina intègre un système robuste de gestion des erreurs d'authentification :

- **Login incorrect** : Message générique "Email ou mot de passe incorrect" pour des raisons de sécurité.
- **Tentatives de connexion successives** : Après 5 échecs, le compte est verrouillé pendant 15 minutes.
- **Réinitialisation du mot de passe** : Lien "Mot de passe oublié" avec token sécurisé valable 24 heures.
- **Rafraîchissement du token JWT** : Renouvellement automatique via refresh token.
- **Session expirée** : Redirection vers la page de connexion.
- **Demande de resaisie** : Bouton "Réessayer" et lien "Créer un compte".

3.2.4 Gestion de la perte de messages et alternatives réseau:

- **Timeout des requêtes** : 30 secondes avec possibilité de réessayer.
- **File d'attente des requêtes** : Stockage local et retransmission automatique.
- **Système de retry** : 3 tentatives avec délai exponentiel (1s, 2s, 4s).
- **Stockage local** : Persistance des données de session dans localStorage.
- **Reconnexion automatique WebSocket** : Toutes les 5 secondes.
- **Barre de statut de connexion** : Affichage en temps réel de l'état.
- **Notification de reprise** : Toast "Connexion rétablie. Les données ont été synchronisées."

3.2.5 Intégration du chatbot avec système RAG:

La plateforme intègre un système RAG (Retrieval-Augmented Generation) qui combine la recherche d'informations contextuelles avec la génération de réponses par un LLM.

Architecture RAG :

- **Base de connaissances** : Documents PDF, rapports médicaux, guides utilisateur, politiques du CHU Ibn Sina.
- **Système d'indexation** : Découpage des documents en chunks et indexation vectorielle .
- **Moteur de recherche vectorielle** : Similarité cosinus pour trouver les chunks pertinents.
- **Générateur de réponses** : LLM LLaMA 3.2 via Ollama.

Fonctionnement du chatbot MIRA :

1. L'utilisateur pose une question via l'interface du chatbot.
2. La question est transformée en embedding vectoriel.
3. La base vectorielle est interrogée pour trouver les chunks les plus similaires.
4. Les 5 chunks les plus pertinents sont extraits avec leurs métadonnées.
5. Le prompt est construit avec les chunks comme contexte.
6. Le LLM génère une réponse basée sur le contexte fourni.
7. La réponse est affichée avec les références des sources utilisées.

3.2.6 WebSocket pour le suivi en temps réel:

Les WebSockets sont utilisés pour :

- La messagerie instantanée entre utilisateurs.
- Le suivi en temps réel des livraisons.
- Les notifications en direct.
- Le chat de support.

3.2.7 Outils de développement:

- **Maven** pour le backend (gestion des dépendances).
- **npm** pour le frontend.
- **Vite** pour le serveur de développement frontend.
- **Alembic** pour les migrations de base de données.
- **OpenAPI Swagger** pour la documentation API.
- **Logs** pour le diagnostic.
- **Docker Compose** pour l'orchestration.

3.3 Implémentation de la plateforme E-Santé Ibn Sina:

3.3.1 Mise en place du back-end:

Le backend utilise FastAPI avec une architecture en couches :

- **Controllers** : Exposent les endpoints REST.
- **Services** : Portent les règles métier.

- **Repositories** : Gèrent l'accès aux données (SQLAlchemy).
- **Models** : Représentent les tables (33 entités).
- **Schemas** : Validation des données via Pydantic.

Principaux endpoints :

Endpoint	Description
/api/auth/login	Authentification utilisateur
/api/auth/register	Inscription patient
/api/auth/refresh	Rafraîchissement du token JWT
/api/auth/logout	Déconnexion
/api/patients/{ipp}/symptoms	Saisie des symptômes
/api/patients/{ipp}/diagnosis	Obtention du diagnostic IA
/api/doctors/{id}/appointments	Gestion des rendez-vous
/api/doctors/{id}/prescriptions	Gestion des prescriptions
/api/secretaries/appointments/validate	Validation des rendez-vous
/api/pharmacists/prescriptions/{hash}/verify	Vérification des ordonnances
/api/deliveries/missions	Gestion des missions de livraison
/api/admin/users	Gestion des utilisateurs
/api/admin/stats	Statistiques
/api/ai/mira	Assistant MIRA
/api/ai/amie	Analyse des symptômes AMIE
/api/ai/medctm	Analyse d'images MedCTM
/api/ai/triage	Triage IA
/api/ai/matching	Matching IA

3.3.2 Mise en place du front-end :

Le frontend React est organisé autour de routes protégées :

Route	Interface
/login	Page de connexion
/register	Page d'inscription
/dashboard/patient	Tableau de bord patient
/dashboard/doctor	Tableau de bord médecin
/dashboard/secretary	Tableau de bord secrétaire
/dashboard/pharmacist	Tableau de bord pharmacien
/dashboard/delivery	Tableau de bord livreur
/dashboard/admin	Tableau de bord administrateur
/patient/symptoms	Saisie des symptômes
/patient/appointments	Gestion des rendez-vous
/patient/prescriptions	Consultation des ordonnances
/patient/deliveries	Suivi des livraisons
/patient/payment	Paiement en ligne
/doctor/consultation	Espace de consultation
/doctor/anatomy	Visionneuse 3D anatomique

Suite de la table

Route	Interface
/doctor/calculations	Calculateurs cliniques
/secretary/appointments	Gestion des rendez-vous
/secretary/schedule	Gestion des emplois du temps
/pharmacist/prescriptions	Vérification des ordonnances
/delivery/missions	Gestion des missions
/admin/users	Gestion des utilisateurs
/admin/logs	Logs d'audit
/admin/settings	Paramètres
/ai/mira	Assistant MIRA

3.3.3 Mise en place de l'authentification :

L'authentification repose sur JWT avec :

- **Endpoints publics** : /api/auth/login, /api/auth/register, /api/auth/refresh, /api/auth/logout .
- **Stockage du token** : localStorage.
- **Protection des routes** : ProtectedRoute côté frontend, @PreAuthorize côté backend .
- **RBAC** : 6 rôles distincts (Patient, Doctor, Secretary, Pharmacist, DeliveryMan, Admin).

3.3.4 Intégration du QR Code :

Le QR Code est généré pour les ordonnances sécurisées :

- **Génération** : Sous forme d'image base64.
- **Encodage** : Identifiants chiffrés de la prescription.
- **Validation** : Vérification en pharmacie pour délivrance.
- **Sécurité** : Authentification unique et marquage "délivrée".

3.3.5 Intégration du suivi en temps réel :

- **WebSocket** : Configuration pour la messagerie et le suivi.
- **Leaflet** : Cartes interactives pour le suivi GPS.
- **Suivi des livreurs** : Position en temps réel sur carte.
- **Notifications** : Mises à jour en direct des statuts.

3.4 Présentation des interfaces de l'application :

Voici les captures d'écran des principales interfaces de la plateforme E-Santé Ibn Sina, avec leurs descriptions fonctionnelles.

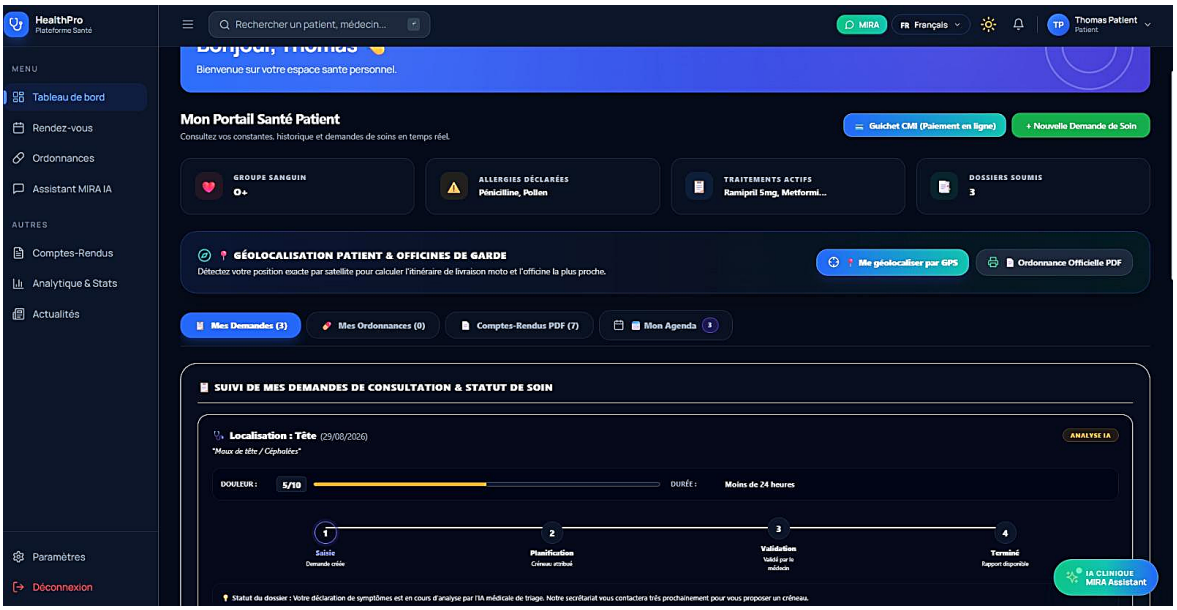

Figure 3.1: Tableau de bord patient - Vue d'ensemble

Objectif principal : Offrir au patient une autonomie complète dans la gestion de son parcours de soins (demande de soins, prise de rendez-vous, suivi de traitement, paiement en ligne et réception des ordonnances).

Fonctionnalités clés :

- **Formulaire de Demande de Consultation avec Triage IA** : Saisie guidée des symptômes avec analyse automatique en temps réel par le module de Triage IA pour déterminer le degré d'urgence.
- **Prise de Rendez-vous en Ligne** : Consultation des disponibilités des praticiens et réservation directe sur l'agenda avec possibilité d'exportation vers Google Calendar ou fichier ICS.
- **Consultation & Impression d'Ordonnances Sécurisées** : Visualisation des prescriptions médicales et impression avec QR Code d'authentification unique.
- **Suivi en Direct des Livraisons de Médicaments** : Carte interactive (`LiveDeliveryMap`) permettant de suivre le livreur en temps réel depuis la pharmacie jusqu'au domicile.
- **Géolocalisation des Établissements & Pharmacies** : Détection GPS pour localiser les centres de santé et pharmacies de garde à proximité.
- **Paiement en Ligne Sécurisé** : Module de paiement bancaire intégré (CMI Payment Modal) pour le règlement des consultations ou des livraisons.
- **Assistant IA MIRA (Co-pilote Santé 24/7)** : Chatbot conversationnel basé sur l'architecture RAG/LLM pour répondre aux questions de santé du patient et l'orienter.

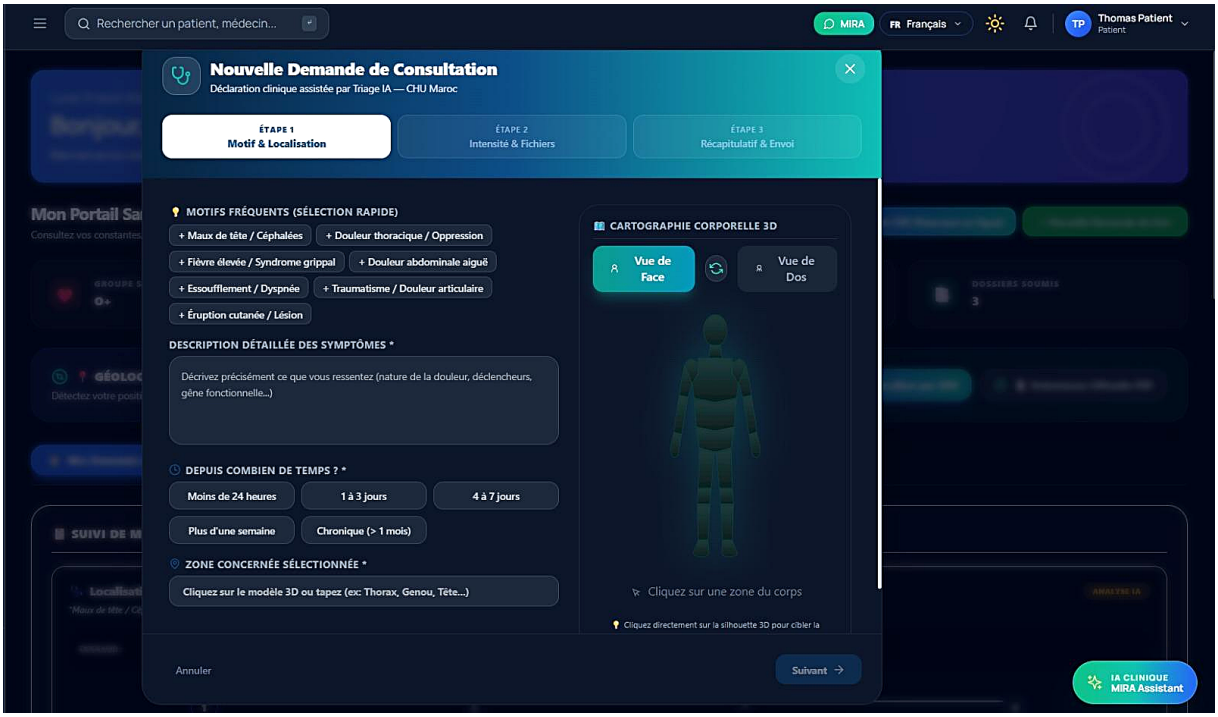

Figure 3.2: Demande de consultation du patient avec triage IA

Cette interface permet au patient de saisir ses symptômes (texte et photo) avec analyse automatique en temps réel par le module de Triage IA pour déterminer le degré d'urgence. Le patient peut ensuite prendre rendez-vous en ligne en consultant les disponibilités des praticiens.

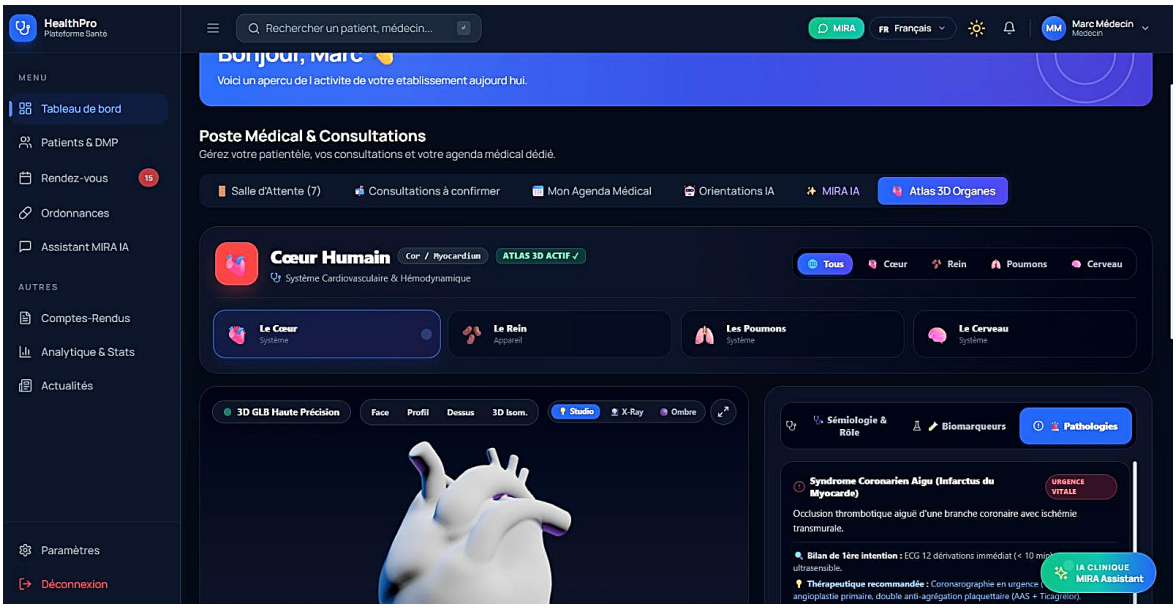

Figure 3.3: Tableau de bord médecin - Espace de travail clinique

Objectif principal : Fournir aux professionnels de santé un espace de travail clinique centralisé pour gérer les consultations, établir les diagnostics, prescrire des traitements et analyser l'anatomie des patients.

Fonctionnalités clés :

- **Gestion de la Salle d'Attente Virtuelle & File de Rendez-vous** : Liste en temps réel des patients du jour avec indicateurs de priorité issus du Triage IA.

- **Espace de Consultation Clinique** (`ConsultationWorkspace`) : Interface d'examen médical permettant la saisie des motifs, l'encodage des diagnostics (norme CIM-10), la prescription d'examens et l'émission d'ordonnances médicamenteuses.
- **Visionneuse Anatomique 3D Interactive** (`AnatomyViewer3D`) : Exploration 3D en WebGL (Three.js) du corps humain et des organes (cœur, poumons, reins, cerveau) avec repères anatomiques interactifs.
- **Module de Calculs Cliniques Instantanés** : Calculateurs médicaux intégrés (IMC, Cockcroft-Gault pour la fonction rénale, Score de Glasgow GCS, Surface Corporelle BSA).
- **Validation / Refus des Demandes de Consultation** : Module de revue des propositions de rendez-vous suggérées par l'algorithme de matching IA.
- **Agenda Dynamique** (`InteractiveCalendar`) : Visualisation et édition des plages horaires d'ouverture et de consultation.
- **Assistant IA MIRA (Aide au Diagnostic)** : Consultation en direct du LLM local pour obtenir des recommandations thérapeutiques ou vérifier des posologies.

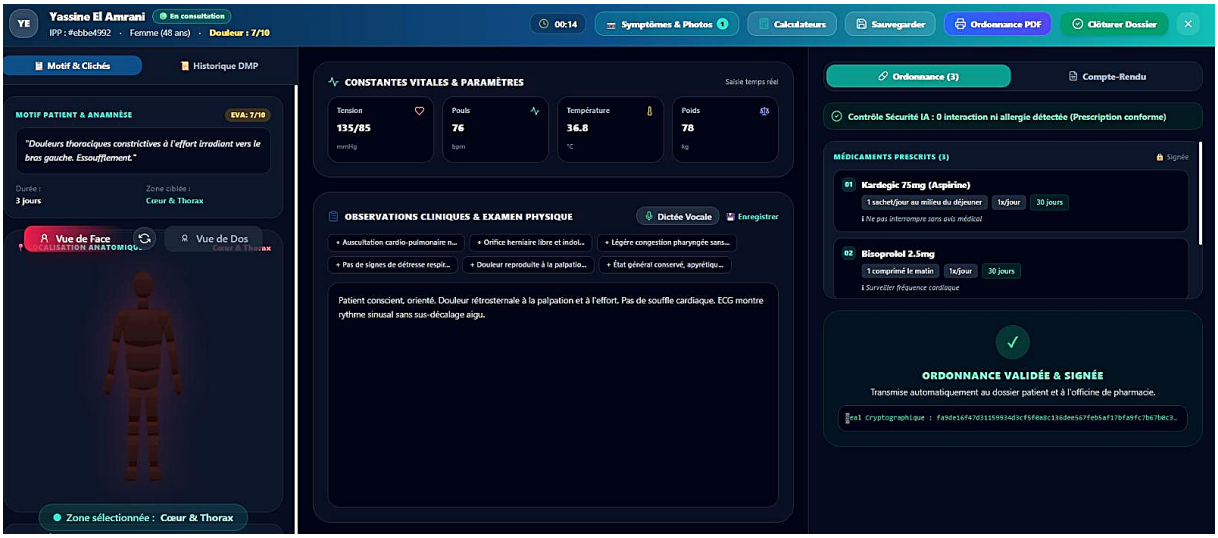

Figure 3.4: Espace de consultation clinique du médecin

L'espace de consultation permet au médecin d'examiner le patient, d'encoder les diagnostics (norme CIM-10), de prescrire des examens, de générer des ordonnances sécurisées avec signature numérique et QR Code, et d'utiliser les calculateurs cliniques instantanés (IMC, Cockcroft-Gault, Score de Glasgow, BSA).

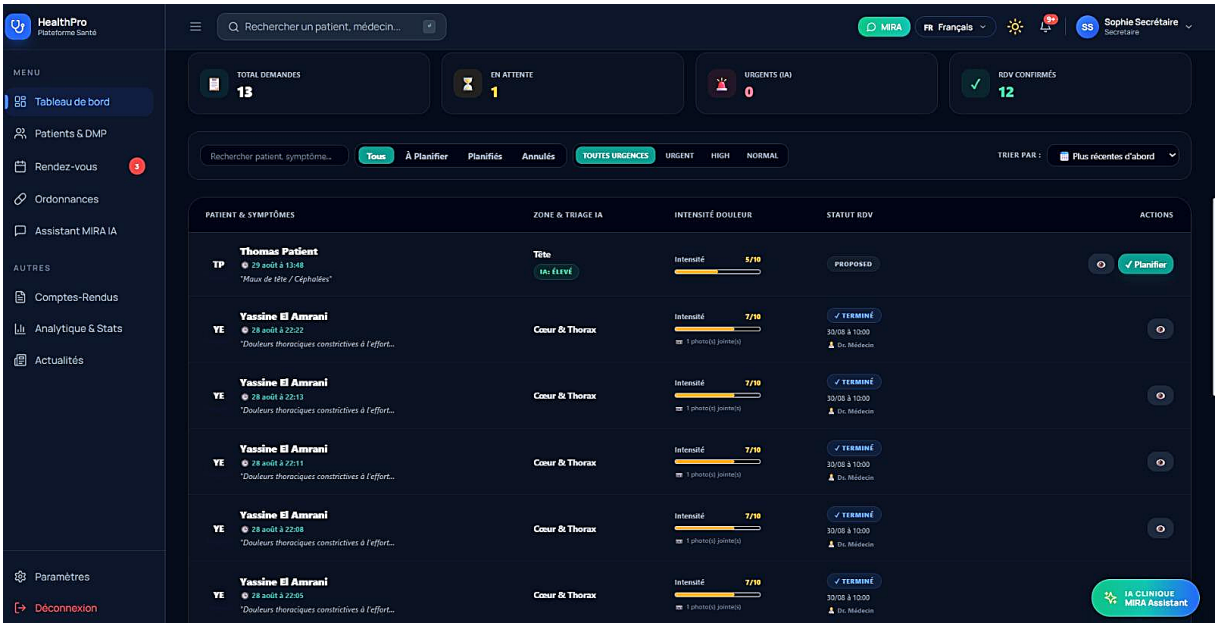

Figure 3.5: Tableau de bord secrétaire - Gestion administrative

Le secrétaire gère les inscriptions, les rendez-vous, les urgences et les statistiques d'activité. Il peut créer et modifier l'emploi du temps des médecins, valider les rendez-vous, et auditer les accès aux dossiers.

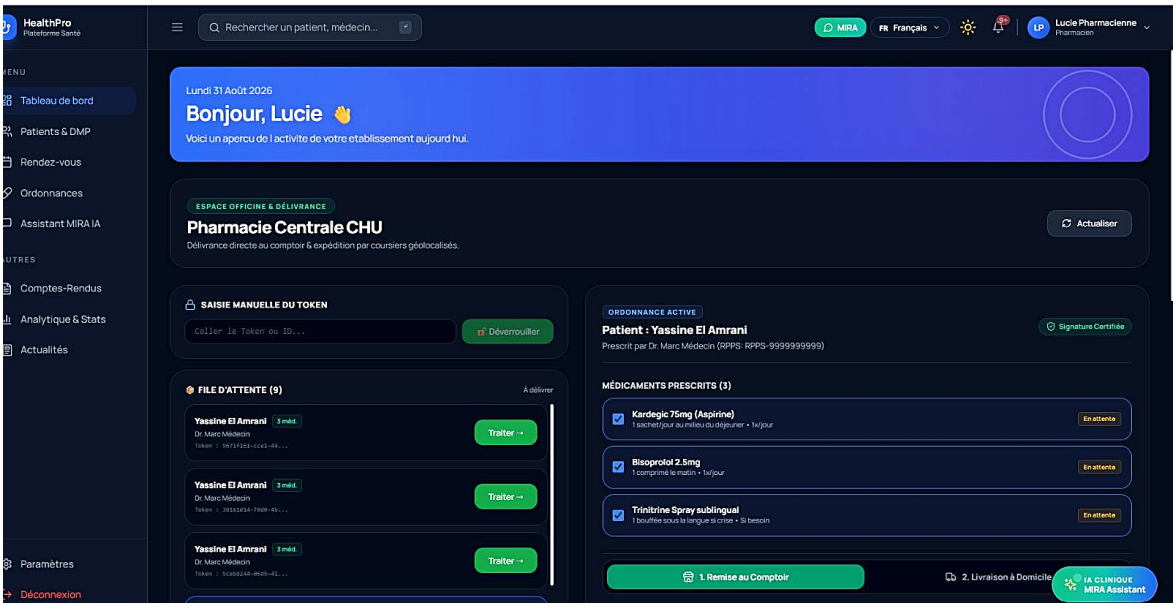

Figure 3.6: Tableau de bord pharmacien - Gestion des ordonnances

Le pharmacien vérifie l'authenticité et l'intégrité des ordonnances via blockchain et SHA-256, délivre les médicaments, et reçoit des alertes d'interaction médicamenteuse. Il peut consulter l'historique pharmaceutique des patients.

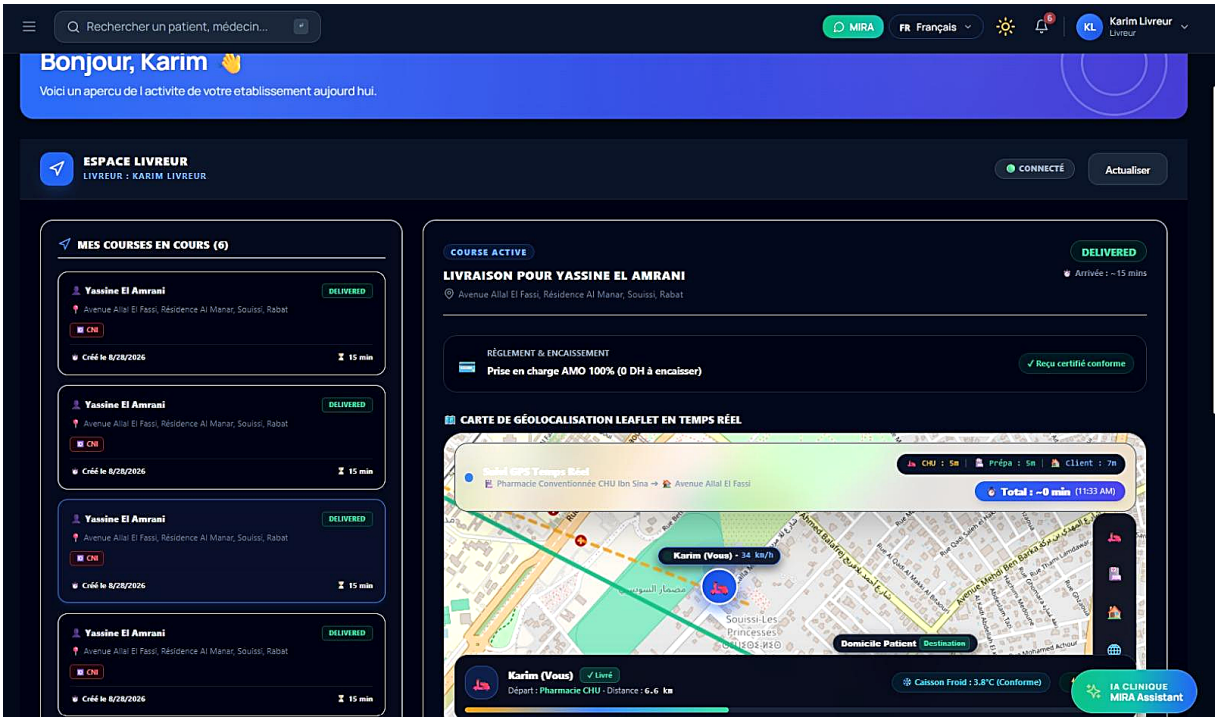

Figure 3.7: Tableau de bord livreur - Gestion des missions

Le livreur gère ses missions de livraison, communique avec les autres profils, consulte les annonces, et valide les livraisons par QR Code. Sa position est suivie en temps réel pour permettre au patient de visualiser l'avancement de sa livraison.

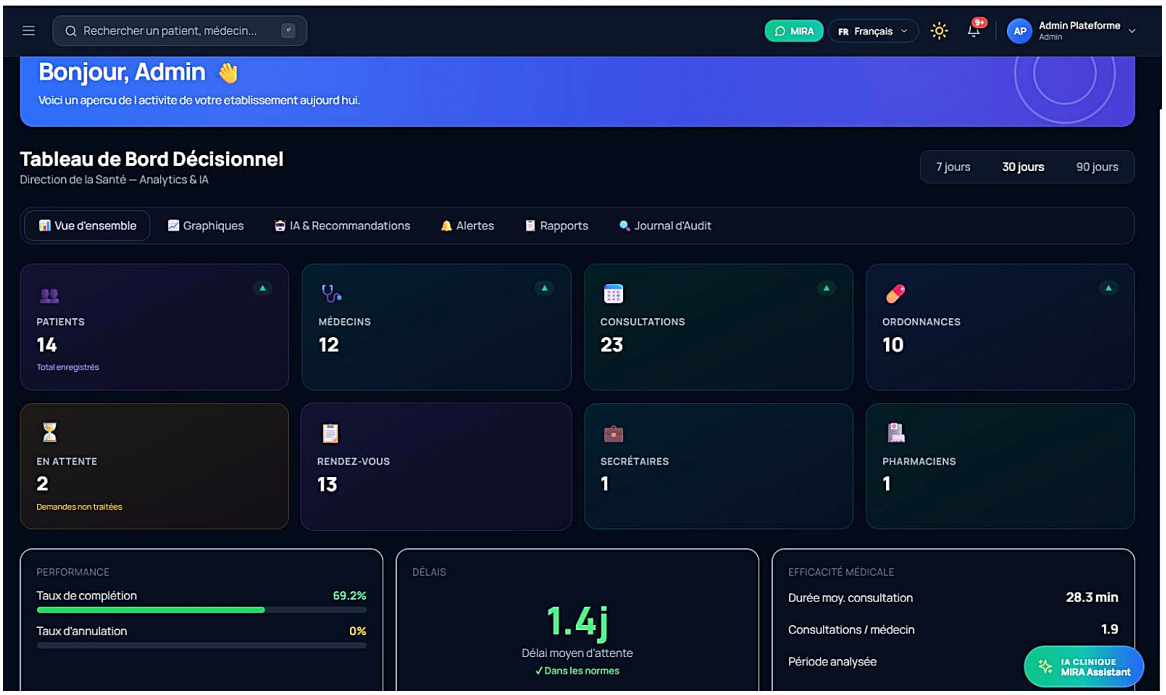

Figure 3.8: Tableau de bord administrateur - Supervision

L'administrateur supervise l'ensemble de la plateforme. Il gère les utilisateurs, active/désactive les API, configure les paramètres, gère les sauvegardes et consulte les logs d'audit pour garantir la sécurité et la traçabilité.

3.5 Tests fonctionnels :

3.5.1 Ordre des tests :

Les tests fonctionnels suivent l'ordre suivant :

1. Démarrer PostgreSQL, Redis, le backend, le frontend et Ollama.
2. Vérifier la connexion avec les comptes de test.
3. Vérifier les redirections par rôle.
4. Créer une demande de consultation en tant que patient.
5. Valider la demande en tant que secrétaire.
6. Assigner le rendez-vous à un médecin.
7. Vérifier la création du rendez-vous.
8. Connecter un médecin.
9. Réaliser la consultation.
10. Créer une prescription.
11. Signer et hacher l'ordonnance.
12. Transmettre à la pharmacie.
13. Vérifier l'ordonnance en pharmacie (blockchain, SHA-256).
14. Délivrer les médicaments.
15. Créer une mission de livraison.
16. Tester le suivi GPS.
17. Tester le QR Code de livraison.
18. Tester les modules IA (AMIE, MIRA, MedCTM).
19. Tester le paiement en ligne.
20. Tester le chat et les notifications.

3.5.2 Tests de gestion des consultations :

Vérifier :

- Création d'une demande de consultation.
- Analyse des symptômes par IA (AMIE, MedCTM).
- Détermination du niveau d'urgence (MIRA).

- Validation par le secrétaire.
- Assignation à un médecin (matching IA).
- Création du rendez-vous.
- Modifications selon les disponibilités.
- Consultation clinique.
- Création de prescription.

3.5.3 Tests de sécurisation des ordonnances :

Vérifier :

- Génération de l'ordonnance par le médecin.
- Signature numérique.
- Hachage SHA-256.
- Ancrage blockchain.
- Transmission au pharmacien.
- Vérification d'authenticité.
- Vérification d'intégrité.
- Délivrance des médicaments.
- Marquage "délivrée".
- Traçabilité complète.

3.5.4 Tests de livraison et validation QR Code :

Vérifier :

- Création d'une mission de livraison.
- Acceptation par un livreur disponible.
- Suivi GPS en temps réel.
- Génération du QR Code de livraison.
- Scan unique.
- Blocage du scan si déjà scanné ou expiré.
- Confirmation de livraison.
- Notification au patient et au pharmacien.

Conclusion générale et perspectives :

Bilan du projet :

Le projet **E-Santé Ibn Sina** a permis de développer une plateforme web complète de gestion centralisée des soins au sein du Centre Hospitalier Universitaire Ibn Sina, répondant aux besoins identifiés. Les objectifs fixés ont été atteints : création d'une plateforme fonctionnelle et sécurisée, gestion complète du parcours de soin, coordination efficace entre les six acteurs, intégration de fonctionnalités avancées (IA, géolocalisation, QR Code, blockchain, messagerie temps réel) et interfaces adaptées à chaque rôle.

Difficultés rencontrées :

Les principales difficultés ont concerné :

- L'assurance de la cohérence entre le frontend et le backend.
- L'alignement des noms de champs et des endpoints.
- La gestion des statuts des rendez-vous et des prescriptions sans transition invalide ;
- L'intégration des services IA avec des temps de réponse acceptables.
- La mise en œuvre de l'architecture RAG pour l'assistant MIRA.
- La sécurisation des ordonnances via blockchain et SHA-256.
- L'interopérabilité via FHIR.

Apports du projet :

Ce projet a permis de mettre en pratique l'ensemble des compétences acquises :

- Conception d'une architecture full-stack (FastAPI, React, PostgreSQL).
- Mise en place de la sécurité JWT et RBAC.
- Développement d'API REST asynchrones.
- Gestion de rôles complexes (6 acteurs).
- Intégration de cartes GPS (Leaflet) pour le suivi en temps réel.
- Génération et validation de QR Code.

- Communication en temps réel via WebSocket.
- Intégration de services IA et RAG (LLaMA 3.2, Ollama).
- Sécurisation des données avec blockchain et SHA-256.
- Interopérabilité via FHIR.
- Méthodologie de test et validation (Pytest, Vitest).

Perspectives :

Plusieurs axes d'amélioration sont envisagés pour enrichir la plateforme :

- Développement d'une **application mobile** pour les patients et les livreurs.
- **Optimisation avancée des itinéraires** de livraison avec algorithmes de tournées .
- Ajout de **notifications push** pour les alertes et rappels.
- Enrichissement du module bénéficiaire et du suivi des patients.
- Extension des fonctionnalités IA avec des modèles plus spécialisés.
- **Intégration avec le système d'information hospitalier** (SIH) existant.
- **Télémédecine** avec consultations vidéo intégrées.
- **Tableaux de bord avancés** avec Power BI pour le pilotage.
- **Archivage permanent** des données via IPFS.

Références bibliographiques :

1. Documentation officielle FastAPI - https://fastapi.tiangolo.com/
2. Documentation officielle React - https://react.dev/
3. Documentation officielle TypeScript - https://www.typescriptlang.org/
4. Documentation officielle Vite - https://vitejs.dev/
5. Documentation officielle PostgreSQL - https://www.postgresql.org/docs/
6. Documentation officielle Docker - https://docs.docker.com/
7. Documentation officielle JWT - https://jwt.io/introduction
8. Documentation officielle Leaflet - https://leafletjs.com/
9. Documentation officielle Three.js - https://threejs.org/
10. Documentation officielle Ollama - https://ollama.ai/
11. Documentation officielle LLaMA - https://ai.meta.com/llama/
12. Documentation officielle FHIR - https://hl7.org/fhir/
13. Documentation officielle Tailwind CSS - https://tailwindcss.com/docs
14. Code source du projet E-Santé Ibn Sina : backend FastAPI, frontend React, migrations SQL et fichiers de configuration.